Problem-agnostic hierarchies, path space volume, and log speed-up in reinforcement learning

Tyler Foster[1] and Abdullah N. Malik[2]

[1]Unaffiliated
[2]Tallahassee State College, Tallahassee, FL, USA

May 29, 2026

Abstract

This document proves that for a very general class of reinforcement learning (RL) problem, one can induce non-canonical hierarchical structures on the RL problem that induce a logarithmic speed-up in training time, *even if the RL problem has no obvious human-readable subtask decomposition*. The central insights behind these results is that states in hierarchical RL do not need to have any real-life, executable meaning whatsoever. They can be pure abstractions, as long as they lift to executable tasks in environment. The theorem is deliberately stated as an idealized search-complexity theorem rather than as an unconditional training-time theorem. This document accompanies a PyPl package developed by the author, called `state_collapser`. The results in this document are the strongest version currently justified by the package design.

Authorship and LLM assistance. The original RL/counterpoint problem, the path-space geometry motivation, the quotient-task viewpoint, the dynamic tower construction, the nested state/action partition data structure, the coarsening/Young diagram interpretation, and the decisions about which mathematical claims this paper is trying to make are Tyler Foster and Abdullah Malik's work. The design documents and engineer-continuity reports in the `state_collapser` and `HGraphML` repositories consistently record Foster and Malik as the source of the substantive mathematical direction and of the critical corrections that shaped this research document and its accompanying repositories.

This document was prepared with substantial assistance from OpenAI GPT-5-series/Codex systems, referred to in this collaboration as GPT-5.4/5.5. The LLM contribution includes drafting, rewriting, TeX organization, local consistency checks against the repository, citation and BibTeX assistance, and turning Foster and Malik's notes and corrections into definitions, propositions, algorithms, proof sketches, corollaries, and explanatory prose. Several formal statements and proofs are therefore LLM-assisted formulations of Foster/Malik-originated ideas.

The LLM should not be credited as an independent discoverer or verifier of the mathematical program. It functioned as a drafting and engineering assistant under Foster and Malik's direction. The claims in this document remain research claims: they require ordinary mathematical review, experimental benchmarking, and future revision.

All figures were created by Tyler Foster in `draw.io` and `LaTeX`, with one instance of GPT-image assistance: Figure **??** includes a 3-armed, red tubular surface. That surface is a GPT-image enhanced of a line drawing that Foster did.

Contents

1 Introduction

1.1 Logarithmic speed-up from hierarchical structure

Hierarchical reinforcement learning (*HRL*) is a broad class of strategies in *reinforcement learning* (*RL*) that attempt break hard RL problems down into some sort of structured hierarchy of more tractable RL problems [PSTQ22]. The intuition here is the same as the intuition behind binary search algorithms: decomposing the addresses involved in a computation along a geometric hierarchy can introduce a computational speed-up that comes from the logarithmic behavior of the addresses in the hierarchy.

Example 1.1.1. Binary search, HNSW, and dictionary traversal.

Both *binary search* [CLRS09] and *hierarchical navigable small worlds* (*HNSW*) [Kle00] [FGK+09] [MY16] provide logarithmic speed-up to search algorithms by implementing geometric addressing, as in Figure 1.

For binary search, the address of an element in a sorted array can be specified by repeatedly choosing a half-interval. Each comparison discards one half of the remaining candidate addresses, so after q comparisons the unresolved address set has size at most $n/2^q$. Thus locating one element among n possibilities requires only $O(\log n)$ address decisions rather than $O(n)$ sequential checks.

HNSW implements an analogous address system geometrically: long-range edges at high levels give coarse addresses for a region of the point cloud, and lower levels refine that address by shorter local moves. Search therefore proceeds by descending through a hierarchy of increasingly fine neighborhoods, so the number of graph moves needed to localize a target scales logarithmically under the usual navigability assumptions.

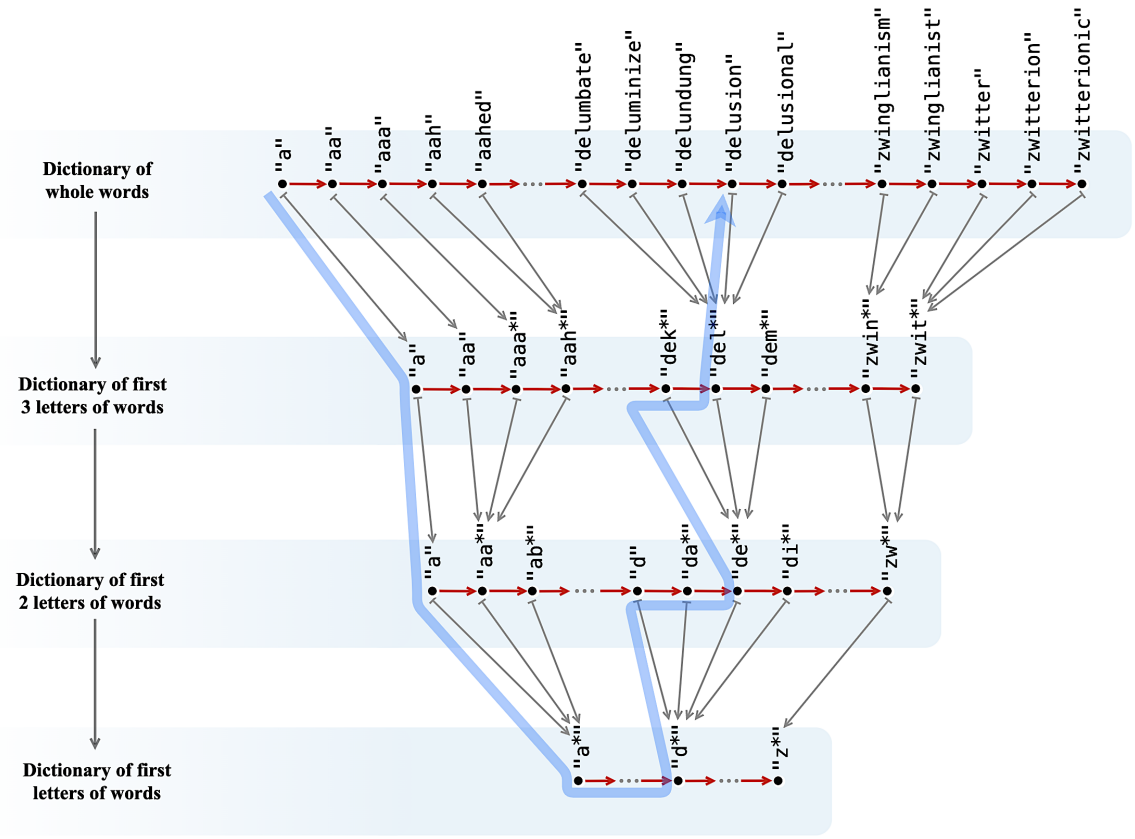


Figure 1: Movement through a dictionary as a path endpoint problem organized by a prefix tower. Prefixes are quotient addresses.

We can also cast the logarithmic speed-up in terms of path specification. The hierarchy contracts long flat paths into short address paths. A path that would require specifying many primitive transitions in the base space can instead be specified by a small number of coarse choices followed by local refinements. The logarithm appears because each coarse address step represents a fixed-scale contraction of the remaining traversal distance.

Abdul Malik applied these ideas to graph ML learning in [Mal24]. In Malik's work, a graph tower is used to organize simplex detection by passing from the original graph to a sequence of quotient graphs. Candidate simplex structure can then be searched for at coarser levels first and lifted back to finer levels only when necessary. The tower therefore acts as an addressing scheme for combinatorial structure in the graph, reducing the amount of flat graph search required before a promising local configuration is identified.

Example 1.1.2. U-Nets and ResNets.

This intuition, that putting a hierarchical structure on a computing problem can induce a logarithmic speed-up, is also hidden in the CNN architecture of U-Nets [RFB15] this idea is only 11 years old, was SOTA at that time, and now it's completely classical. Mind blowing pace of movement in this field![Yeah! Totally agree!] and ResNets [HZRS15], where it again induces a logarithmic improvement.

Indeed, the speed-up provided by HNSW comes from an geometrically thinned hierarchy of coarsened spaces. The original HNSW paper [Kle00], Kleinberg describes the hierarchy as links separated by characteristic distance scale, enabling logarithmic complexity scaling. A U-Net does

essentially the same thing: each downsampling layer reduces the number of sites and increases the physical scale represented by one feature. Thus, a U-Net is HNSW-like for spatial/frequency information flow.

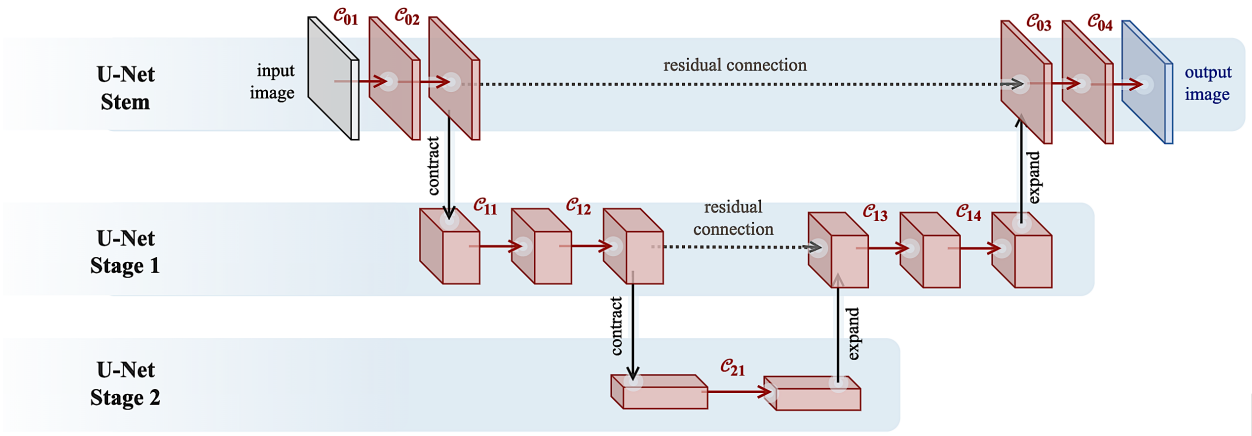


Figure 2: A U-Net-style multiscale architecture is best read here as an instantaneous/tangent analogue of a path space quotient tower

For residual correction, it is *multigrid*-like, in the sense of [AZHE26]. MgNet Do you mean this [HX19] article? Ok yeah I'm a little lost... Does the 2026 paper use a framework developed in the 2019 paper? My understanding is the both introduce a kind of gradient scaling between successive tiers of the U-Net, and uses these scaling factors to say mroe concrete things about the computational efficiency of the U-Net. Let's investigate this more carefully. makes this correspondence especially explicit: restriction maps pass information to a coarser grid, smoothing operations remove local high-frequency error, and prolongation maps lift coarse corrections back to the fine scale. Read through this lens, a residual CNN is not merely a stack of feature extractors; it is also an iterative correction scheme in which coarse representations control the search for fine-scale residual structure.

When the training procedure/loss is organized coarse-to-fine, the U-Net becomes HNSW-like for weight-space search as well, and we get a logarithmic speed-up in train time.

Example 1.1.3. Hints from hierarchical RL. Consider the RL problem of training a program to write Western tonal counterpoint, specifically to write an *a cappella* musical texture formed from n voices, by choosing each next chord in a sequence of n-note chords. So for instance, in a string trio, $n = 3$ and the voices are a violin, a viola, and a cello. In a barbershop quartet, $n = 4$ and the voices are the lead, the tenor, the baritone, and the bass.

Regardless of what the specific voices are, the combinatorics of next-chord choice explodes as the number of voices grows:

1. If the texture has one voice, then the agent is choosing a next note in a melody. If we restrict attention to locally plausible diatonic motion, for example repeated note, step up/down, and small skip up/down, then a reasonable working estimate is

$$\mathbb{E}[\#\,\text{possible next notes}] \;\approx\; 5$$

   If we understand melody writing as making a choice of "next note" over and over again, RL-style, then the number of possible melodies quickly explodes. For instance, with the above working estimate
2. If the texture has two voices, then it is choosing a next interval, subject to voice-leading constraints like "no parallel fifths" and "no crossing voices". As a rough local estimate, if each

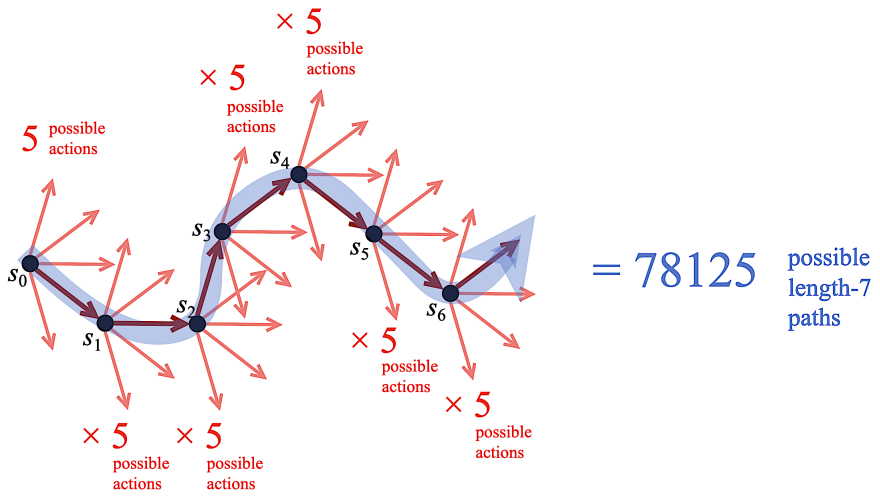


Figure 3: Making a choice among "the next 5 possibilities" 7 consecutive times leads to 78 125 possible sequences of choices.

voice has about five plausible next pitches before cross-voice filtering, then there are about $5^2 = 25$ raw next intervals; after excluding voice crossing, spans over two octaves, and parallel fifths or octaves, a reasonable working estimate is

$$\mathbb{E}[\#\,\text{possible next intervals}] \;\approx\; 15$$

3. If the texture has three voices, then it is choosing a next trichord, subject to voice-leading constraints. The raw local count is about $5^3 = 125$, but the three pairwise voice relations impose ordering, width, and parallel-perfect-interval exclusions; a useful rough estimate is

$$\mathbb{E}[\#\,\text{possible next trichords}] \;\approx\; 60$$

4. If the texture has four voices, then it is choosing a next tetrachord, subject to voice-leading constraints. The raw local count is about $5^4 = 625$, and the six pairwise voice relations cut this down substantially, but not nearly enough to make the flat action space small; a conservative working estimate is

$$\mathbb{E}[\#\,\text{possible next tetrachords}] \;\approx\; 180$$

The number of plausible next actions grows very quickly with the number of voices, and the number of possible episodes grows exponentially with the length of the musical passage. Thus even before the reward function is subtle, the flat path space for "next chord" counterpoint is already enormous.

The agent is not merely choosing among a few next moves; it is searching through a huge space of possible voice-leading paths.

In practice, human musicians do not usually learn or write counterpoint by trying all voices at once until a good texture appears. They build the object hierarchically. One first learns to write a good melodic line. Then one learns to add a good accompanying line against it. Then one learns to fill in inner voices, decorations, suspensions, or harmonic support against an already constrained two-part skeleton. Each stage reduces the effective search problem for the next stage. Fixing or learning a melody first quotients out much of the full multi-voice texture space: the later voices are no longer searched against the entire path space of all possible chords, but against a structured lower-dimensional scaffold. The hierarchy is not just a musical convenience. It is an address system for path space, as depicted in Figure 4.

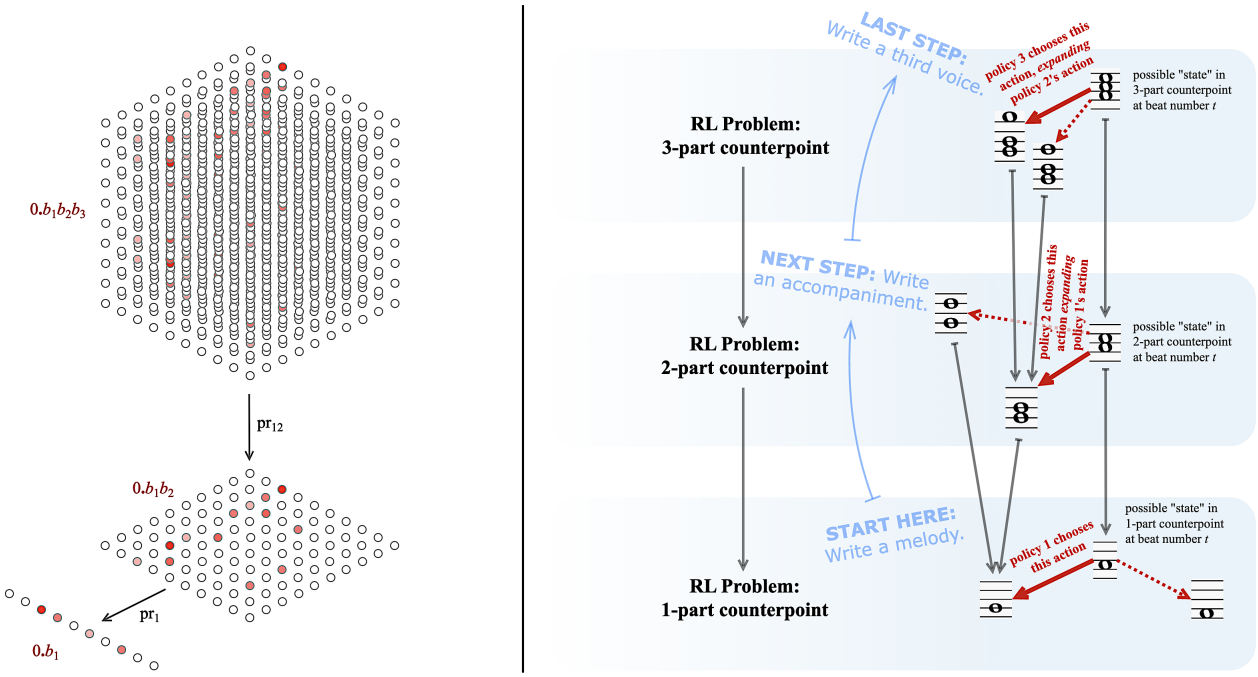


Figure 4: (Left) A positional address system replaces flat enumeration by logarithmic-length descriptions. (Right) Counterpoint as staged path-space search: one-part melodic structure constrains two-part writing, which then constrains richer textures. The package does not require such a human-readable hierarchy, but this example explains why quotient hierarchy matters.

Example 1.1.4. Difficulties in RL for coordinated movement. Coordinated movement problems are difficult because the useful action is rarely a single independent primitive. The agent must move several degrees of freedom in concert, and the reward signal often appears only after a coordinated path has been completed. This creates a flat search problem with a large combinatorial branching factor. A quotient tower helps by grouping coordinated families of primitive moves into abstract movement directions, so the agent can search over coarse coordinated addresses before refining to concrete actions.

For example, suppose a robot arm must move a box into a container in three-dimensional space. The flat state/action graph records all small coordinated motions of the arm, gripper, and box. But one quotient task is obtained by projecting away horizontal motion and retaining only height: first learn how the box should move in the z-direction as a function of time. Another quotient task is obtained by projecting to the floor: learn where the box must move in the xy-plane, temporarily ignoring its height. These projected problems are not subtasks in the sense of being independently executable pieces of the full task. They are quotient tasks: they collapse some degrees of freedom so that the control problem has a shorter address. The full policy can then be learned as a refinement that lifts these quotient policies back to coordinated motion in the original configuration graph.

Question 1. *Why should the contractions in the quotient tower be required to be ideal? As suggested by the graph HNSW speed in simplex search achieved in [Mal24], could one obtain the same logarithmic search effect from random or approximately balanced contractions, provided that reward descent and liftability hold only approximately?*

1.2 Main theorem: Problem agnostic, hierarchical log speed-up for RL train

Suppose that the agent's explored state/action graph admits a quotient tower whose levels behave like an address system for the useful paths in the problem. Instead of searching directly through a flat set of primitive paths, the controller first chooses a coarse quotient address and then refines that choice through successive tiers until an executable primitive path is obtained. The theorem below records the idealized consequence of this situation: when the tower covers the relevant path space, rewards descend well enough through the quotients, and lifts back to executable actions are available, the flat search term can be replaced by a logarithmic address-search term plus the residual costs of lifting, refinement, and statistical estimation.

Assumptions 1.2.1. *Rewards are action-local after any necessary state augmentation, and quotient rewards are trained or estimated as direct-image rewards (sum and max?). Sum? as per Eq (3) of [HBML22]. The max is probably taken over all quotients, as in Eq (4) of [HBML22]. Is that even practical?. For both equations, I imagine there is a derivation using Statistics. I think the right think to do here is clarify that the specific computational meaning of "direct image" is variable here. Basically any reasonable theory of aggregation works, with sum, weighted sum, RMS, and max being reasonable examples.*

There are positive integers $b_m, b_{m-1}, \dots, b_0$ and nonnegative residual terms ϵ_i such that the tower controller can choose the coarsest address and each subsequent refinement with expected cost $O(\log b_i) + \epsilon_i$, with induced address capacity $\prod_{i=0}^{m} b_i \geq \#\{\text{discovered states}\}$. When the tower is approximately balanced, the product is of the same order as $\#\{\text{discovered states}\}$. A small explanation on what 'balanced' means loosely would be helpful here. Perhaps by referring to numbers in the cumulative reward kernel modulo transition properties

Theorem 1.2.2. *When Assumptions 1.2.1 hold, the tower controller's search cost $\mathrm{Cost}_{\mathcal{T}}(\Omega)$ has a logarithmic upperbound:*

$$\mathrm{Cost}_{\mathcal{T}}(\Omega) \quad \leq \quad C \sum_{i=0}^{m} \log b_i \; + \; \sum_{i=0}^{m} \epsilon_i$$

for a model-dependent constant C. This provides a logarithmic improvement to flat search over Ω, which has enumeration cost $\mathrm{Cost}_{\mathrm{flat}}(\Omega) = \Theta(|\Omega|) = \Theta(\mathcal{P}\text{-Vol}_\Omega)$, where $\mathcal{P}\text{-Vol}_\Omega = |\Omega|$ is the finite path volume of the relevant path set. In the balanced case, where $\prod_i b_i \asymp |\Omega|$, this becomes

$$\mathrm{Cost}_{\mathcal{T}}(\Omega) = O\big(\log \mathcal{P}\text{-Vol}_\Omega\big) + \sum_{i=0}^{m} \epsilon_i.$$

Aaaah ok I think my poor informal wording in this Theorem and its setup has created a lot of confusion for you later in the paper about the path space. ...I think it's kind of obscured what it is I'm up to with the path space, so let me explain it differently here.

First, you should be thinking of $\mathbf{Path}(H)$ as "something *waaaaay* worse/more complicated than the fundamental group $\pi_1(H, *)$ (for some choice of node $*$ in H). The fundamental group $\pi_1(H, *)$ is hard to understand in general. It is "loops at $*$ modulo homotopy," i.e., $\mathbf{Loop}(H, *)/\sim$. Loops at $*$ are just paths that have start and end point $*$. So the space $\mathbf{Path}(H)$ is way bigger than the insane set that we quotient in order to get the fundamental group. All of this is to say that the path space $\mathbf{Path}(H)$ is sort of unknowable. The *path space* is the set of all paths *of all lengths* in H. The *length-0* paths are the same things as nodes. The *length-1* paths are just edges. All paths of length ≥ 2 contain more than one edge.

The relevance to RL is that a "point" $\gamma \in \mathbf{Path}(H)$ amounts to a history in H, i.e., a sequence of consecutive nodes (states) connected by edges (primitive actions). So the reason for even thinking

about $\mathbf{Path}(H)$ in the context of RL is that $\mathbf{Path}(H)$ is the moduli space of things that RL training needs to search over and optimize.

It is *very much* like the path formulation of QM, where reality is somehow optimizing over all paths to minimize this physical quantity S ("action" in a different sense than we're using here). In RL, it's not this physical quantity S we're trying to minimize. Rather, we're trying to maximize a quantity called "total reward," i.e., total payout if we take a given history through state space.

The real problem we run into is that because RL is trying to optimize over this moduli space of histories $\mathbf{Path}(H)$, we have to actually do some kind of geometry on $\mathbf{Path}(H)$. We can't just treat it like a set. The path formulation of QM runs into this exact problem. In QM, it is formulated as "the problem of summing over all Feynman diagrams." It's really an attempt to take an integral, roughly of the form

$$\int_{\mathrm{Path}(TX)} L(\gamma)\, d\gamma,$$

where $\mathrm{Path}(TX)$ denotes all histories through phase space, and where "$L(\gamma)$" is some physical quantity you can compute, given a particular history γ.

Mathematicians look at that integral and think "Are you joking? That's just symbols. That's not describing a real calculation." The shock that

Kontsevich's big achievement around "*stable maps* and "*virtual fundamental classes* was to show that using algebro-geometric tools, you can build something *like* $\mathrm{Path}(TX)$, but much more controlled, namely a moduli space of stable curves, where integration makes sense, in the form of an intersection theory. I am conflating some thigns here, and being loose with ideas, but it's the correct general picture. [CDGP91] [Kat92] [Kon95]

So one of the big moves in the paper below is scraping *just enough geometry* out of $\mathbf{Path}(H)$, in the form of this growing "volume power series" $\mathcal{P} - Vol$ to show that... let me say it this way: If you think about binary search / HNSW a lot like some CS people do, you get this strong sense that "logarithmic addressing speeds up computation because it compresses the volume of a space in a logarithmic way." The thing that is less obvious is that a logarithmic compression also happens to this more terrifying space $\mathbf{Path}(H)$, because to say that correctly, we would need some notion of volume in $\mathbf{Path}(H)$. Even when H is finite, $\mathbf{Path}(H)$ isn't, so this need for volume in $\mathbf{Path}(H)$ is real. It is very much a version of needing an integral like $\int_{\mathrm{Path}(TX)} L(\gamma)\, d\gamma$. The trick of using $P - Vol$ is that it pushing the non-fininteness of the integral into a literal series.

...Maybe we should figure out how to fold some of that above into this text itself. Is it good motivation for what we do?

1.3 Main Corollary: The `state_collapser` package

Our main corollary puts our theorem into production. It says that an efficient implementation of our tier-constructing mathematical model can provide logarithmic speed-up for RL training.

1.3.1. The `state_collapser` package. The mathematics below describe a robust, abstract strategy for inducing an HRL structure on RL problems that have no canonical or intrinsic hierarchical structure. We claim, in this document, to see how to induce HRL structures where there aren't any.

To deliver on our claims in a way that engineers can appreciate and can maybe even deploy in production, we've developed the Python package `state_collapser`.

> **Package name:** `state_collapser`
>
> **Package description:** A Python package for inducing hierarchical RL structure on RL problems with *no* obvious subtask decomposition or natural human-readable hierarchy.

Repo link: TYLERSFOSTER/state_collapser

Making no assumption that there is any hierarchy visible in a given RL problem specification, the `state_collapser` package constructs quotient-style tower structure by recursively contracting discovered state/action-graph structure. The result is a runtime layer for coarse-to-fine HRL control that navigates intermediate, state/action graphs wherein states and actions have been "collapsed", and are not be directly executable, but can be refined or "lifted" to executable executable behavior.

The payoff is a reduction in effective search and training burden. Rather than repeatedly learning over the full, "fine-scale" path space associated to the original RL problem, the agent can learn first over more collapsed quotient structure, then return to finer tiers only for the additional correction that the coarser tiers could not supply. In the best case, this replaces broad flat exploration with staged coarse-to-fine control over a much smaller effective path space.

Corollary 1.3.2 (**Problem agnostic, hierarchical log speed-up for RL train).** *If the runtime contractions produced by* `state_collapser` *generate a quotient tower satisfying coverage, action-local reward descent, liftability, and balanced addressability for the path set actually explored by training, then the package replaces the flat path space search term* $\Theta(\mathcal{P}\text{-Vol})$ *by a quotient-tower search term of order*

$$O(\log \mathcal{P}\text{-Vol}) + \textit{lift/refinement/statistical residuals.}$$

2 Underlying formulations of RL and HRL

We will try to follow the notation in [SB18, pp. xix-xxii]. That said, we *will* deviate from that book's notation when it serves out purposes.

2.1 Standard formulation of RL

2.1.1. Three curtains. We want to distinguish three layers that are often conflated in informal discussions of RL.

1. The *real system* or *real world*, which may contain far more structure than the learner ever observes.
2. The *environment model* presented to the agent. These are the state/action interface, transition mechanism, and reward mechanism through which the real system is made computationally accessible.
3. *The agent's internal training representation*, which may replace concrete states by histories, observations, quotient cells, or learned features.

This project exists in the second and third layers. The quotient tower we build is a structured computational representation through which the agent can move more efficiently.

We now fix the formal state/action language used for the environment model. The definitions in this subsection are intentionally elementary: states are nodes, primitive actions are directed edges, and histories are paths. This gives us a concrete graph-theoretic object on which policies, rewards, rollout distributions, and eventually quotient towers can all be defined.

Definition 2.1.2. A *finite directed state/action-graph* H is a finite directed graph

$$H \quad = \quad \mathcal{A}_H \underset{\text{src}}{\overset{\text{tgt}}{\rightrightarrows}} \mathcal{S}_H \tag{1}$$

where $\mathcal{S}_H$ is the set of all possible states of some system we're interested in, and $\mathcal{A}_H$ is the set of all possible primitives actions between any two of the states in our system, meaning that every action in $\mathcal{A}_H$ transpires over a fixed time interval, for time step t to time step $t+1$.

Given a state $s \in \mathcal{S}_H$, we let $\mathcal{A}_H(s) \subset \mathcal{A}_H$ denote the set of all outgoing actions at s, this is, that set of all $\alpha \in \mathcal{A}_H$ such that $\text{src}(\alpha) = s$. In terms of the graph H, we can characterize $\mathcal{A}_H(s)$ as the edge set of the outgoing 1-hop neighborhood of s in the state/action-graph H.

We will return to the following two examples several times. They represent two computational depictions, or encodings of the same physical system, where states are simply positions in some rectilinear region

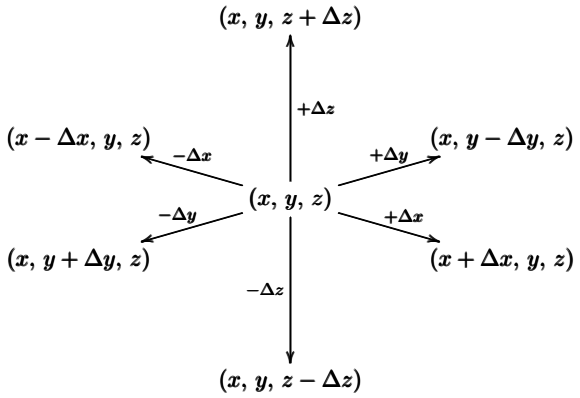
$$\text{Room} \;:=\; [0,\, \ell_1] \times [0,\, \ell_2] \times [0,\, \ell_3] \;\subset\; \mathbb{R}^3,$$

with volume $\text{Vol}(\text{Room}) = \ell_1 \ell_2 \ell_3$

Example 2.1.3. Region of 3-dimensional space as grid lattice: Suppose the RL problem's hidden state space $\mathcal{S}_H$ is just some region in $\mathbb{R}^3$ discretized as a fine grid

$$\mathcal{S}_H \;=\; \Big\{(i\Delta x,\, j\Delta y,\, k\Delta z) \in \mathbb{R}^3 : 0 \leq i \leq N_1,\; 0 \leq j \leq N_2,\; 0 \leq k \leq N_3\Big\}$$

At each interior point, the possible actions are to add or subtract one mesh width unit Δx, Δy, or Δx in the respective direction:



At boundary states, meaning positions on the boundaries of the enclosed region, some subset of these actions will be missing.

One critical feature of this examples is that natural projections pr_{xy}, pr_{yz}, $\text{pr}_{xz} : \mathbb{R}^3 \longrightarrow \mathbb{R}^2$ to coordinates planes and projections pr_x, pr_y, $\text{pr}_z : \mathbb{R}^3 \longrightarrow \mathbb{R}$ to coordinate axes, which come from the ambient geometry in this example, induce natural quotient morphisms of H. For instance

$$\text{pr}_{yz} : \mathbb{R}^3 \longrightarrow \mathbb{R}^2 \;\text{ induces quotient morphism }\; H \twoheadrightarrow H/\{x\text{-axis actions}\}$$

and

$$\text{pr}_x : \mathbb{R}^3 \longrightarrow \mathbb{R} \;\text{ induces quotient morphism }\; H \twoheadrightarrow H/\{y\text{- and } z\text{-axis actions}\}\;.$$

Example 2.1.4. Region of 3-dimensional space as a point cloud: Take the same region $[0,\, \ell_1] \times [0,\, \ell_2] \times [0,\, \ell_3] \subset \mathbb{R}^3$, but now let $\mathcal{S}_H$ be a set of $(N_1+1)(N_2+1)(N_3+1)$ points sampled i.i.d. uniformly from $[0,\, \ell_1] \times [0,\, \ell_2] \times [0,\, \ell_3]$. This sampling gives us a *point density*

$$\rho_H \;=\; \frac{\#\mathcal{S}_H}{\mathrm{Vol}(\mathrm{Room})}$$

Fix the *connectivity threshold*

$$\delta_H \;:=\; \left(\frac{9}{4\pi\rho_H}\right)^{1/3},$$

and declare that there are actions

$$(x_1,\, y_1,\, z_1) =: \mathbf{r}_1 \underset{\alpha_1^2}{\overset{\alpha_2^1}{\rightleftarrows}} \mathbf{r}_2 := (x_2,\, y_2,\, z_2)$$

present in the state/action graph H precisely when

$$\mathrm{dist}(\mathbf{r}_1,\, \mathbf{r_2}) \;\leq\; \delta_H$$

Unlike Example 2.1.3, each of the coordinate plane projections pr_{xy}, pr_{yz}, $\mathrm{pr}_{xz} : \mathbb{R}^3 \longrightarrow \mathbb{R}^2$ and coordinate axis projections pr_x, pr_y, $\mathrm{pr}_z : \mathbb{R}^3 \longrightarrow \mathbb{R}$ will tend not to identify any of our sampled points in $\mathbb{R}^3$, and so will not induce a map to a smaller quotient graph.

Remark 2.1.5. Terminal states. Sutton and Barto are careful to distinguish between arbitrary states and *terminal* states, meaning terminal states s with no outgoing actions. In the present note, we let $\mathcal{S}_H$ denote *all* possible states, including terminal states.

Remark 2.1.6. The state/action-graph is *hidden*. One should understand the state/action-graph as representing the space of all possible configurations our system might end up in. In practice, when confronted with a given system, we initially don't know much about possible configurations the system could be in, and what actions are possible. We map out the possible states of the system and actions between the states through a process of exploration and note taking as we do so.

So the agent traverses the hidden state/action-graph H, recording details of what it sees in memory or in some database as it does so.

Definition 2.1.7. A *history* in H is any graph homomorphism

$$\gamma : \left(\underset{0}{\bullet} \xrightarrow{1} \underset{1}{\bullet} \xrightarrow{2} \cdots \underset{k-1}{\bullet} \xrightarrow{k} \underset{k}{\bullet}\right) \longrightarrow H$$

We will often describe a history γ in H by sequence of states and actions in its image by writing

$$\gamma \;=\; \left(s_0 \xrightarrow{\alpha_1} s_1 \xrightarrow{\alpha_2} \cdots \xrightarrow{\alpha_{k-1}} s_{k-1} \xrightarrow{\alpha_k} s_k\right). \tag{2}$$

We will also refer to a history in H as a *path* in H.

The *path space* of H, denote $\mathbf{Path}(H)$, is the set of all histories, of all lengths, starting at all nodes of H. For a given nonnegative length k, we let $\mathbf{Path}_k(H)$ denote the set of all length-k paths in H, starting at all nodes of H.

Remark 2.1.8. Decompoistions of path space. The path space of H is the discrete analogue of the usual topological path space $\mathrm{Path}(X) := C^0([0,1], X)$ associated to a topological space X

Note that $\mathbf{Path}_0(H) \cong \mathcal{S}_H$, the set of nodes of H, and that $\mathbf{Path}_1(H) \cong \mathcal{A}_H$

Thus the state/action graph H can be recovered as the first two layers of its path space: length-0 paths are states, and length-1 paths are actions. Longer paths should be thought of as higher-order rollout data built from this same state/action grammar.

Purely in their capacity as sets, we have the decomposition

$$\mathbf{Path}(H) \;=\; \coprod_{k=0}^{\infty} \mathbf{Path}_k(H),$$

which we can also write as

$$\mathbf{Path}(H) \quad\cong\quad \mathcal{S}_H \;\sqcup\; \mathcal{A}_H \;\sqcup\; \mathbf{Path}_2(H) \;\sqcup\; \mathbf{Path}_3(H) \;\sqcup\; \cdots$$

We extend the action source and target maps (1) in H to *path* or *history source* and *target* maps

$$\mathbf{Path}(H) \underset{\text{src}}{\overset{\text{tgt}}{\rightrightarrows}} \mathcal{S}_H$$

2.1.9. Control and policy. Following the standard RL convention, a Markov policy on H is a stochastic kernel $\pi : \mathcal{S}_H \longrightarrow \mathcal{P}(\mathcal{A}_H)$ such that the distribution $\pi(-|s)$ is supported on the outgoing action set $\mathcal{A}_H(s)$. Thus the policy assigns to each state a probability distribution on the local fiber of admissible actions above that state. In the quotient constructions below, this support condition is deliberately loosened: an abstract state-cell may use action information pooled across several representatives, so the policy becomes uniformized over a coset rather than tied to one concrete tangent fiber. In this limited sense, the policy has a microlocal flavor: it is a rule on the local outgoing directions at a state, and quotienting replaces pointwise local direction data by direction data attached to a neighborhood or coset.

There are two standard ways to display this dependence, depending on what information the policy is allowed to read. The first is the Markov or last-state version, where the policy sees only the current state. The second is the history-based version, where the policy sees the entire rollout so far, or some observation extracted from it:

- Last state' based':
$$\begin{aligned} \pi_\theta : \quad & \mathcal{S}_H \longrightarrow \mathcal{P}(\mathcal{A}_H \mid \mathcal{S}_H) \\ & s \longmapsto \pi_\theta(- \mid s) \end{aligned}$$
- Full history based:
$$\begin{aligned} \pi_\theta : \quad & \mathbf{Path}(H) \longrightarrow \mathcal{P}\Big(\mathcal{A}_H \Big| \mathrm{out}(\mathbf{Path}(H))\Big) \\ & \gamma \longmapsto \pi_\theta(- \mid \mathrm{out}(\gamma)) \end{aligned}$$

2.1.10. The types of RL problems we consider here. We will consider RL problems that can be formulated in the following way.

> **Basic setup.** An agent moves through the state/action-graph H, occupying a single state $s_t \in \mathcal{S}_H$ at each time step t, and traveling along a single edge $s_t \xrightarrow{\alpha} s_{t+1}$ in the interval from time step t to time step $t+1$, for some action $\alpha \in \mathcal{A}_H$ with source state $\mathrm{src}(\alpha) = s_t$.
>
> The agent is rewarded and/or penalized for its actions as it travels through state space. *Solving* a given RL problem means coming up with some kind of policy that controls the agent's movement through state/action-space, such that text missing

In this paper, such a policy will be represented by a stochastic kernel π that assigns probabilities to admissible next actions, either from the current state alone or from whatever history/observation the learner is allowed to use. When no ambiguity is important, we write this in the Markov form $\pi(-|s)$. When the dependence on rollout history matters, we allow π to be evaluated on a history or on an observation extracted from that history.Hope this is not a naive question: by abstract nonsense, $\mathbf{Path}(H)$ is a collection of edges with well-defined source and target maps. Should we allow a history to be viewed as a single action in the past? I imagine we want to conserve the fact that we are working with a Markov Property? No this is a good question! I would word it a little differently: a point in $\mathbf{Path}(H)$ encodes a sequence of actions in H with source/target conditions: $\mathrm{src}(\alpha_{i+1}) = \mathrm{tgt}(\alpha_i)$. So I would say "$\mathbf{Path}(H)$ is a collection of *collections of* edges with well-defined, *matching-up* source and target maps." Regardless of all that, you're making a really good point: The case where the policy π depends on whole histories is a problem for `state_collapser` because `state_collapser` doesn't yet have that feature, and the reason it's not there is because we need to stay in the Markovian regime for the results in this paper. Figuring out how to push past them is a subject for a future project. So we should probably drop that last sentence, maybe replace it with an explanation about our restricted perspective here.

2.1.11. Cummulative local rewards. Some of our results below concern only movement of the agent through the state/action-graph. For these results, we can ignore the specific structure of the rewards the agent receives.

For our results concerning logarithmic speed-up for RL training, on the other hand, we need to impose conditions on these rewards that ensure they have the ability to "descend" down hierarchies of RL problems. Fortunately for our purposes, the types of rewards we need are very common in RL problems where the policy $\pi(\alpha|s)$ consists of actions sampled stochastically from the output distribution of a backpropagation-trainable model $\mathcal{M}$

Remark 2.1.12. We use the language of stochastic kernels, ie..i.e.,, conditional probabilities, to describe the rewards an agent receives as it traverses an RL environment. All the key ideas in this paper are already at play in the special case that all reward sampling processes are deterministic, so that reward distributions and reward kernels become reward values and reward functions, respectively.

We add this extra layer of complexity because *it more closely resembles the way rewards occur or are implemented in real world RL problems.* In real life, most identifiable states of a system do not come with a deterministic reward. Most rewards in real life depend on a huge number of external parameters to the state of the particular system we're interested in a given RL problem.

Definition 2.1.13 (**Rewards for actions).** By a *reward distribution* r associated to a given action $\alpha \in \mathcal{A}_H$, we mean a probability distribution P_α on the real number line $\mathbb{R}$ and some process that samples from it:

$$r \sim P_\alpha$$

Remark 2.1.14. Reward distributions as reward values. When the distribution P_α is a Dirac delta supported at some point $r \in \mathbb{R}$, we can understand the *reward associated to* α as being the deterministic assignment of the reward value r to the action α:

$$\alpha \mapsto r(\alpha)$$

Definition 2.1.15 (**Action-local reward kernel).** *An* action-local reward kernel *on H is a stochastic kernel, i.e., conditional probability*

$$P_{(-)} : \mathcal{A}_H \longrightarrow \mathcal{P}(\mathbb{R})$$

that associates a probability distribution P_α to every action $\alpha \in \mathcal{A}_H$.

Remark 2.1.16. Rewards kernels as reward functions. When every one of these distributions P_α is a Dirac delta supported at some point $r(\alpha) \in \mathbb{R}$, we can understand the action-local stochastic kernel as a deterministic *action-local reward function*

$$r(-) : \mathcal{A}_H \longrightarrow \mathbb{R}$$

Definition 2.1.17 (Cumulative reward kernel). *A* cumulative reward kernel *on histories in H is a stochastic kernel*

$$P_{(-)} : \mathbf{Path}(H) \longrightarrow \mathbb{R}$$

such that:

1. *The restriction of $P_{(-)}$ to $\mathcal{A}_H \cong \mathbf{Path}_1(H) \subset \mathbf{Path}(H)$ is an action local reward kernel.*
2. *There exists some sequence $w_\bullet = (w_0, w_1, w_2, \cdots, w_{k-1})$ of* decay weights *$w_i \in \mathbb{R}_{\geq 0}$, such that for any history $\gamma = (s_0 \xrightarrow{\alpha_1} s_1 \xrightarrow{\alpha_2} \cdots \xrightarrow{\alpha_{k-1}} s_{k-1} \xrightarrow{\alpha_k} s_k)$, of any finite length k, its associated reward can be computed as the convolution product $r(\gamma) = w_\bullet * r(\alpha_\bullet)$, i.e.,*

$$P_\gamma \quad = \quad \sum_{i=0}^{k-1} w_i \, P_{\alpha_{k-i}}$$

Very interesting! Is there a nice way to compute the weights? Like some sort of Parseval's identity? Oh that's interesting! I never thought about that possibility. The convention I had in mind is a naive one that just fixes an initial decay parameter $0 < \omega < 1$, and then defines $w_i := \omega^i$. That convention comes directly from [SB18]. It's like the most naive thing you could do. ...Yeah it would be interesting to do an FFT during training to try to optimize the decay weights. I have seen things where an FFT is used at the bottom-most stage of a U-Net, in order to optimize a filter applied on the frequency at that stage.

Remark 2.1.18. When every one of these distributions P_α is a Dirac delta supported at some point $r(\alpha) \in \mathbb{R}$, we can understand the action-local stochastic kernel as a *action-local reward function*

$$r(-) : \mathbf{Path}(H) \longrightarrow \mathcal{P}(\mathbb{R}),$$

and the condition that the reward function be cumulative becomes the identity of reward values

$$r(\gamma) \quad = \quad \sum_{i=0}^{k-1} w_i \, r(\alpha_{k-i}).$$

Remark 2.1.19. Note that because $\mathcal{A}_H \subset \mathbf{Path}(H)$, we can interpret a cummulative cumulative reward kernel as some sort of free "weighted convolutive extension of an action-local reward kernel to a reward kernel on full histories."

Since $\mathbf{Path}_0(H) \cong \mathcal{S}_H$ is also included in the full path space, a reward kernel on $\mathbf{Path}(H)$ may also assign values to stationary histories. These can be interpreted as rewards for remaining at a state for one virtual step, or equivalently as rewards attached to formal loop or stay transitions. This does not mean that cycles in the environment are being ignored. Ordinary cycles are still paths built from nontrivial actions; the extra length-0 or stay data merely records the reward contribution of being at a state without taking a primitive move.

2.2 Alternative formulation of RL: *supervised ML with virtual rollout database*

I want to rearrange your picture of RL for a moment.

You say "there is an agent, containing enough of a mind that it can follow some policy that guides its behavior as it wandering around in an environment collecting rewards, modifying its policy where it sees fit."

This makes it sound like a dynamic unsupervised learning problem.

No, I say! It's not that at all! It's just the familiar old supervised ML problem.

2.2.1. Policies as distributions on path space. A policy π_θ assigns to each history, or to a chosen observation of that history, a probability distribution over admissible next actions. The state-based notation $\pi(-|s)$ is only the Markov special case. In general, the model may consume a history γ, an observation of a history, or a quotient-tower summary of a history, and then return a distribution on next admissible actions. Hm.. so my question after 2.1.10 has an extension then: does the quotienting preserve Markov Property? Or is that irrelevant? The induced distribution on path space is therefore policy-dependent even when the displayed notation suppresses this dependence.

$$\mu_{\theta,k} \in \Delta(\mathbf{Path}_k(H))$$

over k-step histories. A training run samples from these distributions and uses the resulting reward-labelled histories to update θ.

Sorry, what does the Δ stand for? I have never taken a course on Markov Processes, so I don't know if that's standard terminology. Sorry!. Scratch that. It's a distribution No no. You were right to catch that. You'll find this interesting: It's has the same meaning you're familiar with. It's an $(N-1)$-simplex, where $N := \#\,\mathbf{Path}_k(H)$!!! Really is is the space of probability distributions on the set $\mathbf{Path}_k(H)$, but that's just

$$\Big\{(c_\gamma)_{\gamma\in\mathbf{Path}_k(H)} : \sum_\gamma 0 \le c_\gamma \le 1 \text{ and } c_\gamma = 1\Big\},$$

which is the defintion of a geometric simplex (as opposed to a combinatorial simplex).

2.2.2. Policies as models-plus-weights on a concrete machine If we want to take a materialist's perspective, our policy is *really* just some familiar old ML model, a DNN, or a transformer, or a U-NET, or whatever. And we train this model in exactly the same way we always do: we repeatedly evaluate the model's performance on some set of tasks, and we adjust the model, in response, via gradient backpropagation.

It's really the same old thing.

The only part of this whole situation that's *really* different from vanilla ML is that for RL, our labelled training data is "stored" in a *virtual rollout database*, and that our strategy for pulling entries from this database is different than for vanilla ML.

Definition 2.2.3 (Virtual rollout database). For a fixed policy π_θ, horizon k, and stochastic kernel $P_{(-)} : \mathbf{Path}_{\le k}(H) \longrightarrow \mathcal{P}(\mathbb{R})$, the *virtual rollout database*, denoted $\mathrm{VRD}_{\theta,k}(H,R)$ is a triple consisting of

1. **Keys set:** Histories will serve as *keys*, so a key is any path $\gamma \in \mathbf{Path}_{\le k}(H)$
2. **Query processor:** Given a key $\gamma \in \mathbf{Path}_{\le k}(H)$, the *query processor* returns the reward distribution
$$P_\gamma \ \in \ \mathcal{P}_{\le k}(\mathbb{R})$$

3. **Value sampler:** Given a key γ and the query processor's returned distribution P_α, the *value sampler* samples a reward value from this distribution:

$$r_\gamma \sim P_\gamma$$

The database is *virtual* in the sense that the triples $(\gamma, P_\gamma, r_\gamma)$ that result from queries to it aren't actually stored anywhere. Instead, the value to return is computed when a query is made. Re: your question mark in the substack: I think 'virtual' is a good name. It has the sense of being there whenever necessary, but still not being a predominant part of the conversation. My first thought was that it gives the impression of something solid that relies on this DB, like containers and stuff Yeah I like that. It does have a containerization kind of feel.

MISSING: together with the policy-dependent query distribution $\mu_{\theta,\leq k}$.

2.3 Formalisms for hierarchical RL

Classical hierarchical RL usually begins by adding temporally extended or semantically meaningful control objects to a Markov decision process. The options framework [SPS99] introduces policies with initiation and termination sets; MAXQ [Die99] decomposes the value function according to a hand-specified task hierarchy; and standard treatments such as [SB18], together with surveys such as [PSTQ22, HBML22], organize HRL around skills, subtasks, abstractions, and temporal decomposition. The construction in this paper shares the broad motivation of reducing long-horizon search by adding hierarchy, but differs in the source of the hierarchy. We do not require a human-specified subtask decomposition, nor do we require abstract states to have an independent real-world interpretation. The hierarchy is produced by quotienting the discovered state/action graph itself.

Remark 2.3.1. Subtask / supertask / component task / abstract task. One of the central insights from [Mal24] is that in lots of graph-structured problems, we can can improve search times by putting the graph into a tower of randomly chosen contractions that functions as a *graph quotient HNSW* query system.

This is the bridge from the graph-search picture to HRL. Instead of asking an engineer to name the subtasks in advance, we let the explored graph generate abstract states by contraction. These abstract states behave like supertasks from the agent's point of view: they are coarse movement addresses that summarize many concrete states and actions, even when they do not correspond to a human-readable component of the environment.

Much of the HRL literature is organized around substructure: subtasks, options, skills, components, or modules that are intended to correspond to recognizable pieces of the original problem. The construction in this paper takes a different viewpoint. A "quotient state" need not be a human-readable subtask at all; it may be no more than an abstract cell obtained by contracting state/action structure. Our hierarchy is not primarily a decomposition into meaningful parts, but a quotient organization of the search space into coarser addresses.

3 Problem agnostic, hiearchical structure in RL

3.1 Dynamic state-space graph

To each history $\gamma \in \mathbf{Path}(H)$ we associate two auxiliary, sequences of nested graphs and decorations.

Definition 3.1.1 (Explored subgraph and viewed subgraph). Given a path $\gamma \in \mathbf{Path}(H)$, which is to say, a history

$$\gamma = \left(s_0 \xrightarrow{\alpha_1} s_1 \xrightarrow{\alpha_2} \cdots \xrightarrow{\alpha_{k-1}} s_{k-1} \xrightarrow{\alpha_k} s_k\right) \quad \text{in} \quad H, \tag{3}$$

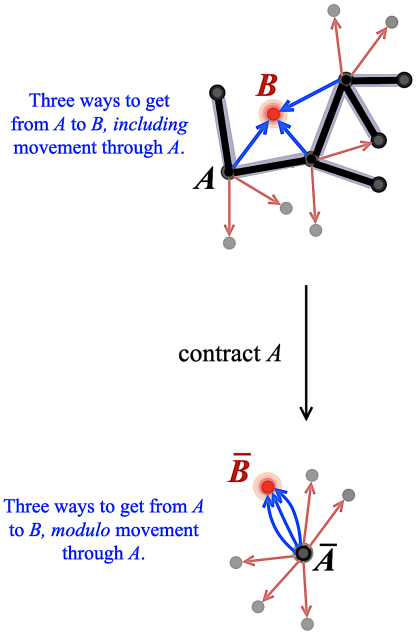


Figure 5: Abstract supertasks as quotient states. A contracted graph cell may not name a familiar physical subtask, but it is meaningful operationally because it gathers concrete states with shared outgoing movement information. From the agent's perspective, choosing the coset is choosing a coarse address for many executable refinements.

for each integer time step $0 \leq t \leq k$, we define the associated *visited subgraph at time* t, denoted H_t^0, to be the subgraph of H formed by all states and actions visited by γ up to and including time step t:

$$H_t^0 \;:=\; \gamma\left(s_0 \xrightarrow{\alpha_1} s_1 \xrightarrow{\alpha_2} \cdots \xrightarrow{\alpha_t} s_t\right) \text{ as s subgraph of } H.$$

Likewise, we define the associated *viewed subgraph at time* t, denoted G_t^0, to be the subgraph of H formed by the union of the visited subgraph H_t^0 at time step t and all outgoing 1-hop neighborhoods $N_1^{\text{out}}(s)$ in H at nodes in H_t^0 at time step t:

$$G_t^0 \;:=\; H_t^0 \cup \bigcup_{s \in \mathcal{S}_t^0} N_1^{\text{out}}(s)$$

We let $\mathcal{S}_t^0$ I am a little confused. We are defining G_t^0 by using $\mathcal{S}_t^0$ – the set of vertices of the graph H – to begin with. Did you mean to write $\mathcal{S}_{G,t}^0$ or something? Or maybe a comment addressing a recycling notation would be helpful Yeah you're right. I like your proposed "$\mathcal{S}_{G,t}^0$." Technically it's not a problem, but it's confusing in current notation. and $\mathcal{A}_t^0$ denote the states and actions in G_t^0, i.e., the nodes and edges of G_t^0, respectively.

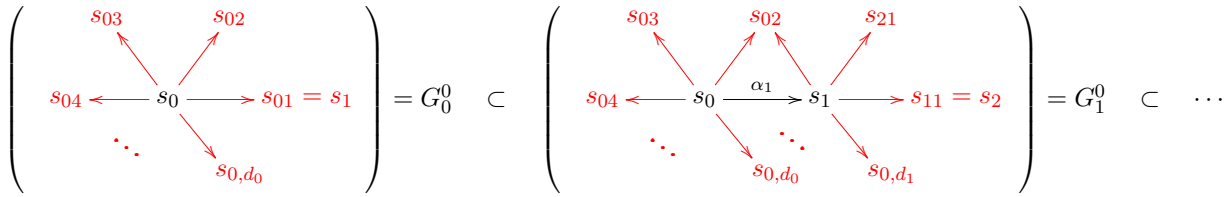
I think a comment on why this is different from the temporal-based subtask in the reviews is warranted? I agree we should add this comment here. We need to be careful here. We need to make sure to be direct about any extent to which temporal-based subtask is categorically dual to our picture. Another important thing here is that there is no learning in our tower. We don't isolate meaningful subtasks... we try instead to create a uniformly collapsed HNSW-type thing. It's important that this is supposed to give a learning speed-up independent of anything specific thing being learned. *We are not claiming that any model is able to learn something that then makes the tower effective. We're claiming, with proof, that the hardcoded table building procedure*

gives speed-up for classical CS reasons, not because we can learn something about it. That's very different than a paper like `https://arxiv.org/pdf/2602.10015`. Are there any papers that show an understanding of the intrinsic/classical source of the speed-up that everyone is discovering in hierarchical learning. Like one thing kind of bold we're saying in this paper is "HRL speed-up is really just n-ary search in disguise, which suggest it has very little to do with intrinsic subtasks."

Remark 3.1.2. By construction, we have chains of graph inclusions

$$(\, s_0 \,) = H_0^0 \;\subset\; (\, s_0 \xrightarrow{\alpha_1} s_1 \,) = H_1^0 \;\subset\; H_2^0 \;\subset\; \cdots \;\subset\; H_k^0.$$

and



Definition 3.1.3 (Decorated viewed subgraph). Given a history as in (3), we decorate each node s of G_t^0, at each time step t, with its set $\mathcal{A}_H(s)$ of outgoing edges *inside* our ambient, hidden state/action-graph H.

Remark 3.1.4. Note that if our state/action-graph H has bounded degree, for instance if we know there are a finite and globally bounded number of degree of freedom to our problem at any given state, then decorating G_t^0 in this way requires a bounded number of adjacent $\mathcal{A}_H(s)$ checks, inducing a linear computational cost

$$\Theta(t), \quad \text{where } t \;=\; \#\text{ time steps.}$$

The implementation should therefore treat the decoration as an incremental local update, not as a repeated scan of the whole explored graph. When a new state is reached, one queries the outgoing edge set at that state once and attaches the result to the node table. If the environment has bounded out-degree, this keeps the discovery cost linear in the number of visited states. The quadratic failure mode would come from rechecking outgoing neighborhoods at every endpoint of every previously discovered edge after each step.

3.2 Dynamic tower of quotient state-spaces

We now contract all edges in G_t^0. However, we do to in a tiered manner, so that we contract edges in stages, eventually exhausting all edges.

We want to allow for both of the possibilities in Examples 2.1.3 and 2.1.4. In the case of Example 2.1.3, the most obvious contraction schema is: (i) contract all x-axis actions, (ii) contract all y-axis actions, (iii) contract all z-axis actions. On a machine, this would mean labelling actions (`"x"`, `"y"`, and `"z"`) and contracting according to label. In the case of Example 2.1.4, the randomness of the edges in 3-space, and the randomness of out-degree from node to node, makes it difficult to contract them in any systematic way. A more natural strategy in this case is to sample 1/3 of the outgoing, not-yet-sampled edges at each node in the original graph, and collapse them all to get next tier.

Although these two contraction schemas are very different, they have the same procedural outline, which we formalize as a "contraction schema."

Definition 3.2.1. Contraction schemas. A *contraction schema* on a graph $G = \mathcal{A} \rightrightarrows \mathcal{S}$ is any explicit or implicit ordered partition of its edge set $\mathcal{A} = \Sigma^1 \sqcup \Sigma^2 \sqcup \cdots \sqcup \Sigma^d$, such that we can retrieve each set Σ^i, one after the other.

3.2.2. Realization in code. On a machine, the contraction schema can take the form of a procedure that i.i.d. uniformly samples some fraction of the edges in the outgoing edge set at each node in G, waits for associated contraction subroutine to complete, and then samples same fraction from remaining arrows, contracting recursively in this way until the graph is completely contracted.

3.2.3. Quotient tower construction: *Mathematical model.* Figure 6 should be read as a top-to-bottom quotient process on the right-hand side. At the top, the hidden state/action-space has been dynamically mapped to the discovered graph $G_t^0 = G_t$, with the current state $s_t^0 = s_t$ and a marked family of edges to contract. The next diagram down is the first quotient G_t^1, obtained after carrying out the first contraction stage; below that is G_t^2, the second quotient. Thus the vertical direction in the figure is not refinement upward from a base graph, but successive contraction downward from the discovered graph to coarser quotient graphs.

3.2.4. Quotient tower construction: *Machine description.* In our mathematical model, we describe a graph G as pairs of maps $\mathrm{src}, \mathrm{tgt} : \mathcal{A} \longrightarrow \mathcal{S}$. Encoding this literal structure, in RAM, would amount to some sort of full adjacency graph encoding.

Because primitive actions in state/action graphs tend to map to a relatively *very* small number of "adjacent" states, these graphs are extremely sparse. We encoded them sparsely, as a node set given by the state space $\mathcal{S}$ and, for each state $s \in \mathcal{S}$, a pointer to the outgoing edge set $\mathrm{Out}(s) := \mathcal{A}(s)$.

We implement the quotient-tower in RAM by maintaining a nested system of labeled partitions of the base state set and base edge set under successive coarsening operations. This places the method near the classical partition-based data-structural viewpoint in algorithms [PT87] and [HPV99]. That said, the construction we now describe is not a *partition refinement algorithm* in the usual sense. Because the operative updates here encode graph edge contractions, they take the form of *coarsenings* of coupled state and action coset partitions. The tower is best understood as a coupled, nested partition system adapted to contraction-driven quotient formation.

The base graph in memory already presents a node table together with an outgoing-edge table, and quotient construction begins by adjoining a state-partition table to this existing graph memory: we define the initial, *tier*-0 *state partition* to be the set of singletons, and we define its associated *tier*-0 *action partition* to be the set of outgoing edge sets

$$\Pi_0^{\mathcal{S}} := \{\{s\} \mid s \in \mathcal{S}_t^0\} \qquad \text{and} \qquad \Pi_0^{\mathcal{A}} := \{\mathrm{Out}(s) \mid s \in \mathcal{S}_t^0\},$$

Note that each of the cells of $\Pi_0^{\mathcal{A}}$ is naturally associated to exactly one of the cells of $\Pi_0^{\mathcal{S}}$, namely its source state I understand that even if there are there sink states in H, it makes no difference to the existence of the map below, since $\mathcal{S}_t^0$ is a viewed subgraph at time t. However, if our stutter convention is to let $s_j \longrightarrow s_j$ be, then $\{s_j\} \subset \mathrm{Out}(s_j)$. I hope it doesn't break anything in the second map below You're right to catch this. We need to think through this.. In this way, me have a natural map

$$\mathcal{A}_t^0 \longrightarrow \Pi_0^{\mathcal{S}}$$

that functions like "a tangent bundle over state space," where the fiber over $\{s\}$ is $\mathrm{Out}(s)$. Likewise, we have a map

$$\Pi_0^{\mathcal{S}} \longrightarrow \Pi_0^{\mathcal{A}}$$

that functions like the "sheaf of tangent spaces over state space" by initializing a pointer at each tier-0 state partition cell $[s]_0 := \{s\} \in \Pi_0^{\mathcal{S}}$ that points to the cell $\mathrm{Out}(s)$.

Inductively, suppose the tier-$(i-1)$ state and action partition tables have already been created. The ith tier is produced by opening fresh tier-i tables initialized from tier-$(i-1)$, then processing every arrow in the contraction block Σ^i. For an arrow $a \xrightarrow{e} b$, we look up the tier-$(i-1)$ cells

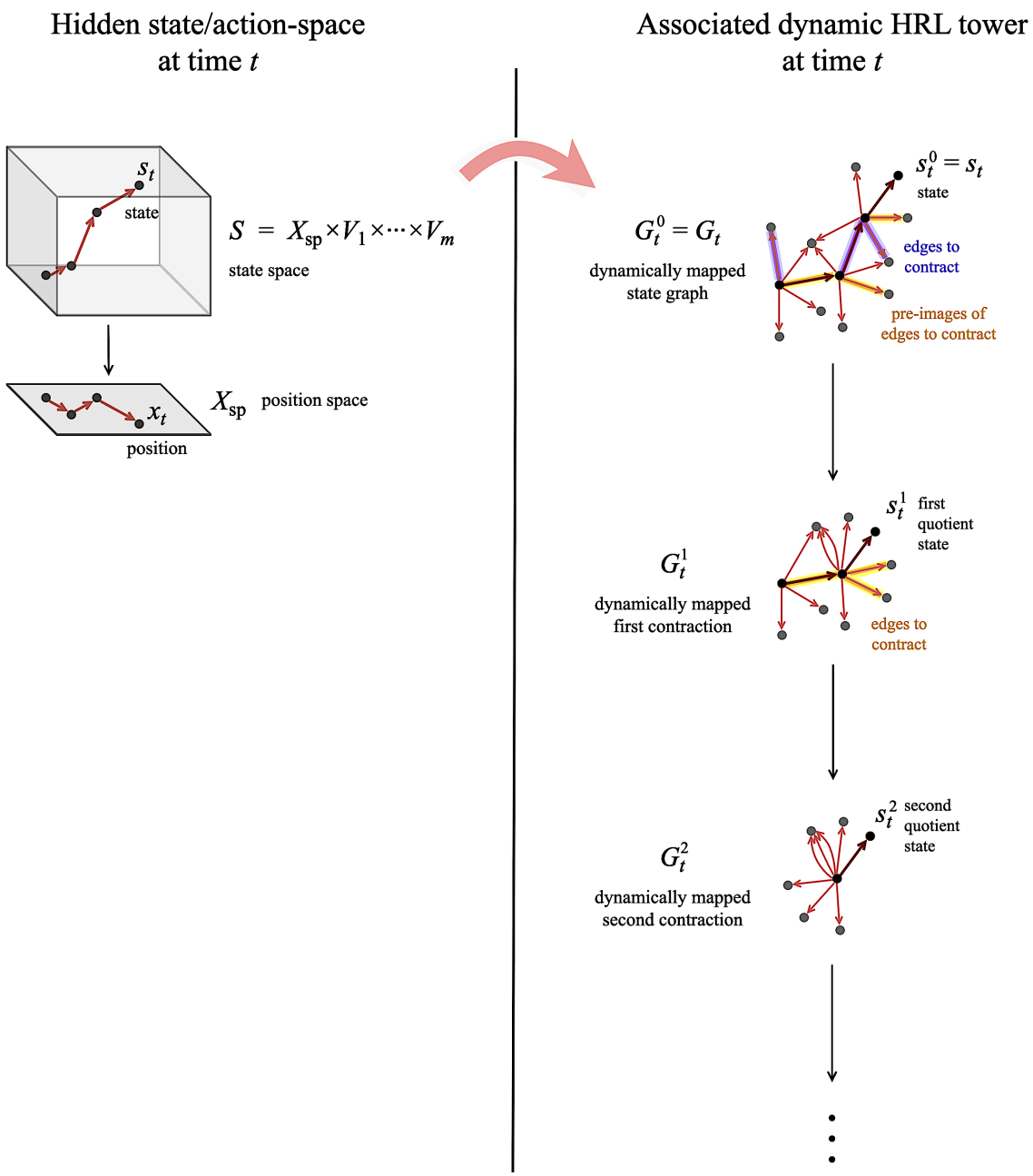


Figure 6: Dynamic HRL tower associated to the hidden state/action-space at time t. The discovered graph $G_t^0 = G_t$ appears at the top of the tower, and successive downward maps contract selected base-tier edges to produce $G_t^1, G_t^2, \ldots$, with the current state tracked through the corresponding quotient states $s_t^0, s_t^1, s_t^2, \ldots$.

containing a and b. If they are distinct, we create a new tier-i state-cell representing their union and a new tier-i outgoing action-cell collection representing the union of their outgoing data, after discarding actions that have become loops inside the merged cell What goes wrong if we don't discard the loops? Are they completely immaterial? Should we say something about a chosen stutter convention here from Definition 4.2.1 I don't know the answer. We need to think through this.. The affected base nodes receive tier-i pointers to the new cell. Repeating this through the block yields G_t^i.

The inductive step from tier-$(i-1)$ to tier i is driven by the ith block Σ^i of the base-edge contraction schema. For each edge $a \xrightarrow{e} b$ in this block, let C_a and C_b be the current state-cells containing its endpoints. If $C_a \neq C_b$, we create a new tier-i state-cell C_{ab} by coalescing C_a and C_b, and we create the corresponding tier-i action-cell data by coalescing their outgoing action-cell collections. Pointers from all nodes in $C_a \cup C_b$ are then initiated to C_{ab}, and C_{ab} is pointed to its merged outgoing-action collection. Repeating this for all arrows in Σ^i gives the next partition layer.

We apply the graph quotienting procedure to this data structure as a sequence of successive parallel coarsening of two coupled partition systems. Assume, by finite induction, that at tier i the *state-side partition table* is a finite, labeled partition

$$\Pi_i^{\mathcal{S}} = \{C_{i,\lambda}^{\mathcal{S}}\}_{\lambda \in I_i^{\mathcal{S}}} \quad \text{of} \quad \mathcal{S}_t^0,$$

and the *action-side partition table* is a finite, labeled partition

$$\Pi_i^{\mathcal{A}} = \{C_{i,\mu}^{\mathcal{A}}\}_{\mu \in I_i^{\mathcal{A}}} \quad \text{of} \quad \mathcal{A}_t^0$$

such that each cell $C_{i,\mu}^{\mathcal{A}}$ lies over a single state partition cell $C_{i,\lambda}^{\mathcal{S}}$, in the sense that there exists a state partition cell $C_{i,\lambda}^{\mathcal{S}}$ in $\Pi_i^{\mathcal{S}}$ such that for all actions $\alpha \in C_{i,\mu}^{\mathcal{A}}$, we have

$$\mathrm{src}(\alpha) \in C_{i,\lambda}^{\mathcal{S}}$$

This ensures that at tier-i, we have a tangent bundle-like map

$$\mathcal{A}_t^0 \longrightarrow \Pi_i^{\mathcal{S}}$$

Each node in our original graph, $s \in \mathcal{S}_t^0$, carries a pointer to its current state-cell $[s]_i \in \Pi_i^{\mathcal{S}}$, and that state-cell carries a pointer to its current outgoing action-cell collection $\mathrm{Out}_i([s]_i)$. Thus we also have a tangent sheaf-like map

$$\Pi_i^{\mathcal{S}} \longrightarrow \Pi_i^{\mathcal{A}}$$

Figure 7 shows the same quotient-tower construction from three compatible points of view. On the left is the graph-contraction picture: selected arrows identify states, internal arrows of a merged cell become loops and are discarded, and the remaining red arrows become outgoing quotient actions. In the middle is the coset-expansion picture: a quotient state is not a new physical state, but a coset of base states, and each representative of the coset inherits the outgoing-action information available from the other representatives. Thus one coarse path through the contracted graph can have several base-level realizations. On the right is the data-structure picture: the tower is stored as nested state Young [Ful97] diagrams[1] together with nested action Young diagrams, with action cells fibered over state cells by outgoing-incidence pointers. The same construction is therefore simultaneously a graph quotient, an operational sharing of outgoing information across cosets, and a coupled coarsening of state/action partition tables in memory.

Concretely, each partition layer can be drawn like a labeled Young diagram. The cells are the blocks of the diagram, and the entries in a cell are the base states or base actions that have been identified at that tier. The tower is then a nested sequence of such labeled diagrams, with cells in one tier sitting inside larger cells in the next. We introduce a bit of extra structure: action diagram is fibered over the state diagram by outgoing-incidence pointers: each state-cell carries a decoration naming the action-cell collection available from that coset, listed as a vertical collection in a given tier.

[1] Young diagram that follows the French convention!

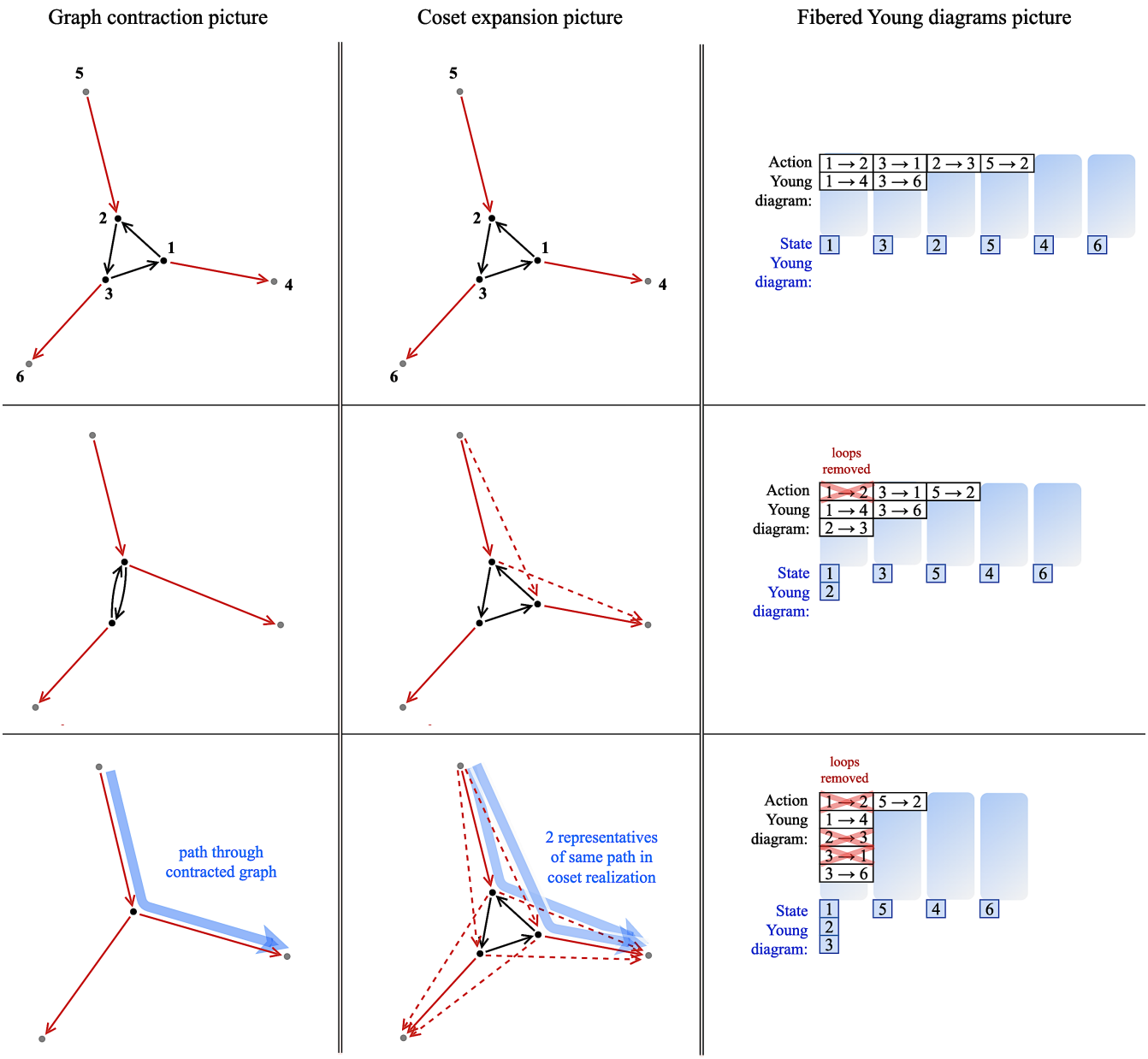


Figure 7: Successive contraction of 3 edges in a graph: (Left) Graph-contraction picture. (Middle) Coset-expanding picture. (Right) Nested, fibered partitions (Young tableaux) structures in RAM encoding the contractions in a light weight, easy to traverse manner.

The tower is nested because these partitions coarsen with i. Each state-cell at tier i lies I don't know if there's an English word that captures $\subseteq$. The word 'lies', I think, conveys $\subset$. Ha. good point... "is or is in" ha ha? inside a unique state-cell at tier $i+1$, and likewise each action-cell at tier i lies inside a unique action-cell at tier $i+1$. Thus the actual tower object is best understood as: (i) the fixed discovered base graph G_t^0, (ii) a nested tower of labeled state partitions $\Pi_0^{\mathcal{S}}, \Pi_1^{\mathcal{S}}, \ldots$, (iii) a nested tower of labeled action partitions $\Pi_0^{\mathcal{A}}, \Pi_1^{\mathcal{A}}, \ldots$, (iv) and, at each tier, the outgoing-incidence pointers from state-cells to action-cells.

The pseudocode below should be read in these terms.In the `require`.. does the state collection

$\mathcal{S}_t^0$ at t not come by default with the graph? Also, what happens if the initial explicity or implicit partion (the schema) isn't given? Should an iid have that done for us? Worth repeating here at the outset IMO. Sorry for making you scroll and not writing down the comments inside the algorithm, and rather leave them here

Shouldn't the function only need the graph itself? If you'd like to keep it, I think in line 1 of the algo, you want $\mathcal{S}_t^0$, not s_t as the input. In Line 19.. is stutter convention very uniform across the field? If yes, I suppose we don't need to explain the word 'chosen' in this line. In Line 26, should the loop not be over $\Pi_{i-1}^{\mathcal{S}}$ instead of $\mathcal{S}_t^0$? I need to think hard about this...

Remark 3.2.5. Traversing the tower An RL agent can traverse this representation of the graph just as well as the adjacency representation, but the sparse representation is easier on memory.

The key point is that traversal in the quotient tower is not the same as taking an edge whose source is literally the current base node. At tier i, the agent is located at a state-cell $[s]_i$, and that cell carries outgoing-action information pooled from all of its representatives. The agent may therefore choose an abstract outgoing action whose concrete representative starts at some other node in the same coset. Execution then requires a lift/refinement step: move within the coset, or select a representative path, so that the abstract action becomes executable in the base graph. This is exactly why the coset must carry unified outgoing-action data. The tower lets the agent jump through quotient addresses first and resolve concrete source compatibility only when lifting back down.

Runtime analysis 3.2.6. Runtime analysis of Algorithm 1 We now analyze the cost of Algorithm 1 as written.

Define $n_t := |\mathcal{S}_t^0|$, $m_t := |\mathcal{A}_t^0|$, and recall that d is the number of blocks in our implicit contracting partition $\mathcal{A}_t^0 = \Sigma_0^1 \sqcup \Sigma_0^2 \sqcup \cdots \sqcup \Sigma_0^d$. It's important here that d is like a characteristic constant of the problem that can be interpreted as the dimension of the problem's underlying state space, so that $|\mathcal{S}_t^0| \sim \ell^d$, where ℓ is some variable representing "the expected length of one dimension of the problem." In other words, we should think of d as being

$$d \;\sim\; \log_\ell |\mathcal{S}_t^0| \;=\; \log_\ell n_t \tag{4}$$

...There's also a natural argument that the expect expected nature of the sparsity in state/action graphs gives us the following expected value of m_t is

$$m_t \;\sim\; n_t\, d \;\sim\; n_t \log_\ell n_t$$

Write $m_i := |\Sigma_0^i|$, so that $\sum_{i=1}^d m_i = m_t$. Note every base-tier edge is processed exactly once by the contraction schedule.

Initialization cost. The initialization phase consists of:

1. creating the singleton state-partition cells $[s]_0$ for all $s \in \mathcal{S}_t^0$,
2. reading off the outgoing-edge buckets $\mathcal{A}_t^0(s)$,
3. and initiating the tier-0 node-to-state and state-to-action pointers.

The singleton initialization costs $O(n_t)$, while reading the outgoing-edge table costs $O(m_t)$ if the adjacency structure is already present in memory, or $O(n_t + m_t)$ if it must be traversed by source. Hence the full initialization cost is

$$T_{\text{init}} = O(n_t + m_t).$$

Algorithm 1 Successive Quotients via Nested State/Action Coset Partitions

Require: Current discovered base graph $G_t^0 = (\mathcal{S}_t^0,\ \mathcal{A}_t^0)$, a current base state $s_t \in \mathcal{S}_t^0$, and an explicit or implicit ordered partition of the base-tier edge set $\mathcal{A}_t^0 = \Sigma_0^1 \sqcup \Sigma_0^2 \sqcup \cdots \sqcup \Sigma_0^d$.

Ensure: A tower $G_t^0 \twoheadrightarrow G_t^1 \twoheadrightarrow \cdots \twoheadrightarrow G_t^N$ in the form of nested state/action coset partitions decorating G_t^0.

function BUILDTOWER(G_t^0, s_t)

Initialization:

Initialize the tier-0 state-partition table by singletons: $[s]_0 \leftarrow \{s\}$ for each $s \in \mathcal{S}_t^0$.

for $s \in \mathcal{S}_t^0$ **do**

Initialize the tier-0 action-side partition table by the outgoing edges: $\mathrm{Out}_0([s]_0) \leftarrow \mathcal{A}_t^0(s)$

Initiate a tier-0 pointer from s to its state-cell $[s]_0$.

Initiate a tier-0 pointer from $[s]_0$ to its outgoing-action cell collection $\mathrm{Out}_0([s]_0)$.

end for

G_t^0 in memory $\leftarrow$ node table on $\mathcal{S}_t^0$ decorated by the tier-0 state and action partition pointers.

Inductive step:

$i \leftarrow 1$

while $i \le d$ **do**

Copy previous partition tables forward as initial tier-i tables: $\Pi_i^{\mathcal{S}} \leftarrow \Pi_{i-1}^{\mathcal{S}},\ \Pi_i^{\mathcal{A}} \leftarrow \Pi_{i-1}^{\mathcal{A}}$

for $a \xrightarrow{e} b \in \Sigma_0^i$ **do**

Let $C_a \in \Pi_{i-1}^{\mathcal{S}}$ be the tier-$(i-1)$ state-cell containing a.

Let $C_b \in \Pi_{i-1}^{\mathcal{S}}$ be the tier-$(i-1)$ state-cell containing b.

if $C_a \neq C_b$ **then**

Coalesce state data into new tier-i state cell: $C_{ab} \leftarrow \mathrm{MergeStateCells}(C_a, C_b)$

Associate tier-$(i-1)$ action cells: $D_a := \mathrm{Out}_{i-1}(C_a),\ D_b := \mathrm{Out}_{i-1}(C_b)$

Coalesce action data into new tier-i action cell: $D_{ab} \leftarrow \mathrm{MergeActionCells}(D_a, D_b)$

Remove from D_{ab} all actions whose source and target both lie in C_{ab}, recording their contribution according to the chosen stutter convention.

for node in $C_a \cup C_b$ **do**

Initiate new tier-i pointer to new tier-i state-cell C_{ab}.

Initiate new tier-i pointer from C_{ab} to new tier-i action cell D_{ab}.

end for

end if

end for

for $s \in \mathcal{S}_t^0$ **do**

Initiate new pointer from node s to its tier-i state-cell $[s]_i \in \Pi_i^{\mathcal{S}}$

Initiate new pointer from node s to its action cell collection $\mathrm{Out}_i([s]_i) \subseteq \Pi_i^{\mathcal{A}}$

end for

G_t^i in memory $\leftarrow$ same node table on $\mathcal{S}_t^0$, but with these pointers to $\Pi_i^{\mathcal{S}}$, $\Pi_i^{\mathcal{A}}$

$i \leftarrow i + 1$

end while

return $(G_t^0, G_t^1, \ldots, G_t^N)$

end function

Per-tier cost. Fix a tier index $i \in \{1, \dots, d\}$. As written, the tier-i work has three parts.

First, the algorithm copies the previous partition tables forward and later re-initiates the node pointers for all states. The explicit final loop in Line 26 already contributes $O(n_t)$ work per tier, regardless of how many contractions actually occur in Σ_0^i.

Second, the algorithm processes the edge block Σ_0^i. For each

$$a \xrightarrow{e} b \in \Sigma_0^i$$

it performs:

1. two state-cell lookups,
2. one state-cell coalescence if the cells differ,
3. one action-cell lookup on each side,
4. one action-cell coalescence,
5. and pointer initiation for all nodes in the merged state-cell.

If we denote by $U_i^{\mathcal{S}}$ the total cost of all state-cell coalescences and their induced node-pointer updates at tier i, and by $U_i^{\mathcal{A}}$ the total cost of all action-cell coalescences at tier i, then the edge-processing loop contributes

$$O(m_i + U_i^{\mathcal{S}} + U_i^{\mathcal{A}}).$$

Third, the algorithm stores the resulting tier-i decorations back into the in-memory graph view, which is again accounted for by the explicit node loop and therefore contributes no asymptotically new term beyond $O(n_t)$.

Accordingly, the total cost of tier i is

$$T_i = O\big(n_t + m_i + U_i^{\mathcal{S}} + U_i^{\mathcal{A}}\big).$$

Total runtime. Summing over all tiers gives

$$T_{\text{BuildTower}} = T_{\text{init}} + \sum_{i=1}^{d} T_i = O(n_t + m_t) + \sum_{i=1}^{d} O\big(n_t + m_i + U_i^{\mathcal{S}} + U_i^{\mathcal{A}}\big).$$

Using $\sum_{i=1}^{d} m_i = m_t$, this simplifies to

$$T_{\text{BuildTower}} = O\left(d\, n_t + m_t + \sum_{i=1}^{d} U_i^{\mathcal{S}} + \sum_{i=1}^{d} U_i^{\mathcal{A}}\right).$$

This is the correct general bound for the pseudocode as stated.

By (4), we can expect this to be

$$T_{\text{BuildTower}} = O\left(n_t \log_\ell n_t + \underbrace{m_t}_{\substack{\text{controlled} \\ \text{by sparsity}}} + \sum_{i=1}^{d} U_i^{\mathcal{S}} + \sum_{i=1}^{d} U_i^{\mathcal{A}}\right).$$

[It's an important point here that almost all real life control problems are sparse, since states can only move to nearby states...]

What depends on the coalescence implementation. The terms $U_i^{\mathcal{S}}$ and $U_i^{\mathcal{A}}$ depend entirely on how the partition tables are implemented.

- If state-cell and action-cell coalescence are implemented naively by explicit member copying and repeated pointer rewriting, then these terms can be large, and in the worst case one can obtain quadratic behavior inside a tier.
- If cell coalescence is implemented by a standard union-by-size or union-by-rank disciplineI think a reference here would be nice to have I agree. I'll try to find one., then within a fixed tier, each base element changes its canonical representative only logarithmically many times. In that case, one obtains the tier-wise bounds

$$U_i^{\mathcal{S}} = O(n_t \log n_t), \qquad U_i^{\mathcal{A}} = O(m_t \log m_t),$$

and therefore

$$T_i = O\big(n_t + m_i + n_t \log n_t + m_t \log m_t\big).$$

Summing over i then yields the worst-case bound

$$T_{\text{BuildTower}} = O\big(d\, n_t \log n_t + d\, m_t \log m_t + d\, n_t + m_t\big).$$

Would one coalescences not be treated as a single step in almost any language? Then $U_i^{\mathcal{S}} = \left|\Pi_i^{\mathcal{S}}\right|$ and $U_i^{\mathcal{A}} = \left|\Pi_i^{\mathcal{A}}\right|$, and the U_i's would depend on i. Ok this is good. I'm going to address this and a comment below at once. So the first important thing to realize is that I am very purposefully trying to draw an analogy between the dimension d_{dim} of a state space and the expected degree d_{deg} in a graph. This is because in a positively-directed grid lattice, we have $d_{\text{deg}} = d_{\text{dim}}$. In a bi-directed grid lattice, we have $d_{\text{deg}} = 2\, d_{\text{dim}}$. The strategy for building the intrinsically meaningful tower for a grid lattice goes "contract x, then contract y, then contract z. As a partition across the whole state space, this looks like $\Sigma^x \sqcup \Sigma^y \sqcup \Sigma^z$. So the general pattern really is that I'm going to treat the number of cells in this partition as something like "the last vestige of dimension I have for a general graph."

The other thing that's going on is that I'm then treating this value d as if it were a logarithm: $d \sim \log_\ell n_t$. This is again coming from the grid lattice case, so we should be really careful about what we're saying here. The idea is that in a grid lattice *in a compact region of* 3*-space*, if each axis has ℓ grid points, then the total number of points in the grid lattice region is ℓ^d. Because we're imagining points in our graph to be states in state space, this is the identity $n_t = \ell^d$, which gives us an expected value $d \sim \log_\ell n_t$.

Interpretation of the bound. This worst-case bound should not be misread as the actual expected runtime burden in the online setting. Two structural facts matter.

First, the ordered partition

$$\mathcal{A}_t^0 = \Sigma_0^1 \sqcup \Sigma_0^2 \sqcup \cdots \sqcup \Sigma_0^d$$

ensures that each base-tier edge participates in the contraction schedule exactly once. Thus the algorithm is not repeatedly rescanning the full edge set at every tier.

Second, and more importantly for the runtime story of the package, Algorithm 1 is the *full-build* description. In the actual online setting, one does not repeatedly rebuild the whole tower from scratch at each time step t. Instead, one uses the incremental update algorithm: if a realized move

produces no new exploration The main while loop in line 10 will still keep on running, though I need to investigate., then

$$G^{\bullet}_{t+1} = G^{\bullet}_{t};$$

and if a realized move does produce new exploration, then only the newly discovered base-tier edges are propagated upward through the existing partition tables. The full-build bound above is therefore the correct static cost of the construction, but it substantially overstates the expected amortized online cost.

Space usage. As written, the algorithm stores:

1. the base node table on $\mathcal{S}^0_t$,
2. the base outgoing-edge table on $\mathcal{A}^0_t$,
3. one state-partition layer per tier,
4. one action-partition layer per tier,
5. and node-to-state / state-to-action pointer decorations at each tier.

Thus the explicit space usage is at least linear in the base graph size and linear in the number of tiers: $O(d\, n_t + d\, m_t)$ up to implementation-dependent constants. With structural sharing between tiers, the effective memory burden can be much smaller, but the displayed bound is the correct straightforward bound for the pseudocode as written.

Conclusion. Algorithm 1 is therefore best viewed as a clear *static specification* of the quotient-tower construction. Its full-build runtime is governed by:

$$O\left(d\, n_t + m_t + \sum_{i=1}^{d} U_i^{\mathcal{S}} + \sum_{i=1}^{d} U_i^{\mathcal{A}}\right),$$

with the precise asymptotics determined by the partition-coalescence implementation. The online package runtime is lighter because the next algorithm replaces wholesale rebuilding by incremental propagation of only newly discovered base-tier edges.

I would maybe use a breakable algorithm Can you explain?

4 Path-volume and logarithmic speed-up.

4.1 Finite-horizon path-volume

For a finite state/action graph G, let $\mathbf{Path}_k(G)$ denote the set of length-k histories in G, and let

$$\mathbf{Path}_{\le K}(G) := \bigsqcup_{k=0}^{K} \mathbf{Path}_k(G)$$

denote the finite-horizon path space through horizon K. Write

$$N_k(G) := \left|\mathbf{Path}_k(G)\right| \qquad \text{and} \qquad N_{\le K}(G) := \sum_{k=0}^{K} N_k(G).$$

If every state had exactly d instead of d for degree, how about a different letter? It would be an nonintuitive choice like p or something (but maybe not b). That's because d already refers to the number of partitions of the actions in the contraction schema See above, where I explain that the conflation is intentional. We should probably explain more clearly, but I do want the notation to stress the analogy outgoing actions, and if every path were admissible, then $N_k(G)$ would grow like d^k, up to the number of allowed initial states. Real RL environments differ from this naive model because of invalid moves, termination rules, policy bias, hard constraints, symmetries, and reward irrelevance. The point of the following definition is to keep both pieces of information visible: the exponential branching model and the actual finite path count.

Definition 4.1.1 (**Path-volume series).** Fix a reference branching number $d \geq 1$. The finite-horizon path-volume series of G at horizon K is

$$\mathcal{P}\text{-Vol}_G(d; K) := \sum_{k=0}^{K} c_k d^k,$$

where

$$c_k := \frac{N_k(G)}{d^k}.$$

Equivalently,

$$\mathcal{P}\text{-Vol}_G(d; K) = N_{\leq K}(G).$$

The coefficients c_k record the deviation between the naive d^k branching model and the actual admissible path count.

The notation is deliberately redundant. The expression $\sum c_k d^k$ remembers the path-space growth intuition, while the equality with $N_{\leq K}(G)$ makes the quantity a finite combinatorial invariant.

Definition 4.1.2 (**Policy-effective path-volume).** Let π_θ be a policy, and let $\mu_{\theta,k} \in \Delta(\mathbf{Path}_k(G))$ be the distribution on length-k histories induced by π_θ, the environment transition rule, and the reward/termination mechanism. For a threshold $\varepsilon > 0$, define the effective support at length k by

$$N_k^{\theta,\varepsilon}(G) := \left|\{\gamma \in \mathbf{Path}_k(G) \,:\, \mu_{\theta,k}(\gamma) \geq \varepsilon\}\right|.$$

The ε-effective path-volume is

$$\mathcal{P}\text{-Vol}_G^{\theta,\varepsilon}(K) := \sum_{k=0}^{K} N_k^{\theta,\varepsilon}(G).$$

An entropy-based variant replaces support size by $\exp(H(\mu_{\theta,k}))$.What is H? I thought it referred to a graph. I don't know what the exponential means here with that distribution. Yeah this is confusing. The idea is that what's "really" happening in RL is that we're selecting from path space with a certain distribution. As an agent learns a policy, it stops seeing all paths/histories equally, It's starts to have strong preferences for what states it even going to explore in the first place.

The raw path-volume is a worst-case geometric object. The policy-effective path-volume is closer to a training diagnostic. Both matter: the first describes the ambient path-space burden, while the second measures the portion of that burden the current learner is actually sampling.

Warning 4.1.3 (**Finite horizon only).** The informal expression $\sum_{k\geq 0} d^k$ is useful as a slogan for path-space explosion, but it is not a finite complexity invariant unless it is truncated or discounted. All statements below use a finite horizon K or an explicitly discounted variant.

4.2 Path addresses induced by the partition tower

Algorithm 1 does not rebuild an independent quotient graph from scratch at each tier. It keeps the discovered base graph $G_t^0 = (\mathcal{S}_t^0, \mathcal{A}_t^0)$ in memory, and constructs two nested partition systems over it:

$$\Pi_0^{\mathcal{S}}, \Pi_1^{\mathcal{S}}, \dots, \Pi_N^{\mathcal{S}} \qquad \text{and} \qquad \Pi_0^{\mathcal{A}}, \Pi_1^{\mathcal{A}}, \dots, \Pi_N^{\mathcal{A}}.$$

The state partitions coarsen the fixed base state set $\mathcal{S}_t^0$, and the action partitions coarsen the fixed base action set $\mathcal{A}_t^0$, with outgoing-incidence pointers from state-cells to action-cells at every tier.

For each tier i, write

$$\pi_i^{\mathcal{S}} : \mathcal{S}_t^0 \longrightarrow \Pi_i^{\mathcal{S}}$$

for the map sending a base state to its tier-i state-cell. Likewise, after choosing the stutter convention used when contracted loops are removed, write

$$\pi_i^{\mathcal{A}} : \mathcal{A}_t^0 \longrightarrow \Pi_i^{\mathcal{A}} \cup \{*_i\}$$

for the map sending a base action to its tier-i action-cell, or to the tier-i stutter symbol $*_i$ if the action has become internal to a state-cell and has been removed from the outgoing action table.

Definition 4.2.1 (**Tier-i path address**). Let

$$\gamma = \left(s_0 \xrightarrow{\alpha_1} s_1 \xrightarrow{\alpha_2} \cdots \xrightarrow{\alpha_k} s_k\right) \in \mathbf{Path}_k(G_t^0).$$

The tier-i address of γ is the path-like word

$$\pi_{i,*}(\gamma) := \left(\pi_i^{\mathcal{S}}(s_0) \xrightarrow{\pi_i^{\mathcal{A}}(\alpha_1)} \pi_i^{\mathcal{S}}(s_1) \xrightarrow{\pi_i^{\mathcal{A}}(\alpha_2)} \cdots \xrightarrow{\pi_i^{\mathcal{A}}(\alpha_k)} \pi_i^{\mathcal{S}}(s_k)\right),$$

with the convention that stutter symbols $*_i$ are either retained as formal stay moves or compressed into the adjacent fiber correction term.

This is the precise sense in which the explicit algorithm still induces quotient maps on paths. The maps are not computed by taking an abstract equivalence-relation hull after the fact. They are read directly from the tierwise state and action partition tables produced by the coarsening procedure.

Definition 4.2.2 (**Path-address image and lift fiber**). Let $\Omega \subseteq \mathbf{Path}_{\le K}(G_t^0)$ be a finite path set relevant to the learning problem. Its tier-i image is

$$\Omega_i := \pi_{i,*}(\Omega).$$

For an address $\eta_i \in \Omega_i$, the tier-i lift fiber is

$$\mathrm{Fib}_i(\eta_i) := \{\gamma \in \Omega \,:\, \pi_{i,*}(\gamma) = \eta_i\}.$$

More locally, the refinement fiber from tier i to tier $i-1$ is the set of tier-$(i-1)$ addresses mapping to a fixed tier-i address under the coarsening map.

Remark 4.2.3. The tower generated by Algorithm 1 is therefore enough for the Path-Volume Theorem 4.4.5. The theorem only needs nested partitions of the finite path set Ω, induced by quotient addresses, and lift fibers between adjacent address levels. The state/action partition tables provide exactly that. The one convention that must be fixed explicitly is what happens to actions that become loops inside a merged state-cell: either they remain as formal stutter actions, or their reward contribution is recorded as a fiber correction when the loop is removed from the outgoing action table.

4.3 Reward descent through action cells

The reward definitions above already allow reward distributions or reward kernels. For the theorem, the relevant operation is the following direct-image construction.:

Definition 4.3.1 (Direct-image reward on an action cell). Let $D \in \Pi_i^{\mathcal{A}}$ be a tier-i action-cell, and let $r : \mathcal{A}_t^0 \to \mathbb{R}$ be an action-local reward function. The direct-image reward of D is

$$r_i(D) := \mathbb{E}\big[r(\alpha) \mid \alpha \in D\big],$$

where the expectation is taken with respect to a chosen reference distribution on the fine actions in D. In a runtime implementation, this reference distribution is the empirical rollout distribution observed in the fiber. For reward kernels, $r_i(D)$ is replaced by the corresponding mixture distribution.

Proposition 4.3.2 (Reward descent by conditional expectation). *Assume rewards are action-local and cumulative. Fix a tier i, and fix a tier-i address*

$$\eta_i = \left(C_0 \xrightarrow{D_1} C_1 \xrightarrow{D_2} \cdots \xrightarrow{D_k} C_k\right) \in \Omega_i.$$

Condition on fine histories $\gamma \in \Omega$ satisfying $\pi_{i,*}(\gamma) = \eta_i$, using the chosen stutter convention. Then

$$\mathbb{E}\big[R(\gamma) \mid \pi_{i,*}(\gamma) = \eta_i\big] = \sum_{j=1}^{k} w_j \, r_i(D_j) + B_i(\eta_i),$$

where $B_i(\eta_i) = 0$ if stutters are retained as formal actions, and where $B_i(\eta_i)$ is the recorded loop/stutter fiber correction if internal loops are removed from the outgoing action table.

Proof. By cumulative action-locality,

$$R(\gamma) = \sum_{j=1}^{k} w_j r(\alpha_j),$$

up to the chosen convention for internal stutter moves. Taking conditional expectation given the tier-i address η_i and using linearity gives

$$\mathbb{E}\big[R(\gamma) \mid \pi_{i,*}(\gamma) = \eta_i\big] = \sum_{j=1}^{k} w_j \mathbb{E}\big[r(\alpha_j) \mid \pi_i^{\mathcal{A}}(\alpha_j) = D_j\big] + B_i(\eta_i).$$

The conditional expectations are exactly the direct-image rewards $r_i(D_j)$. □

Remark 4.3.3. This proposition is the reward-side reason that the current algorithm must record a stutter convention when it removes loops from D_{ab}. Removing loops is harmless for the search address, but their reward contribution must either be retained as a formal stay action or charged to a fiber correction term.

4.4 The path-volume address theorem

Let $\Omega \subseteq \mathbf{Path}_{\le K}(G_t^0)$ be the finite set of reward-labelled histories relevant to the current learning problem. This may be the full finite-horizon path space, an admissible subset, a high-probability support under the current policy, or a benchmark-defined success set.

The flat enumeration cost proxy is

$$\mathrm{Cost}_{\mathrm{flat}}(\Omega) := |\Omega|.$$

When $\Omega = \mathbf{Path}_{\le K}(G_t^0)$, this is exactly the finite-horizon path-volume of Definition 4.1.1.

The tower turns a fine history $\gamma \in \Omega$ into an address chain

$$\pi_{N,*}(\gamma),\ \pi_{N-1,*}(\gamma),\ \ldots,\ \pi_{0,*}(\gamma) = \gamma,$$

where each step refines a coarser address by choosing an element of a lift fiber. This is the quotient-tower analogue of writing an object by its address digits.

Assumption 4.4.1 (Coverage). The path set Ω relevant to the learning problem is represented inside the current discovered graph:

$$\Omega \subseteq \mathbf{Path}_{\le K}(G_t^0).$$

Assumption 4.4.2 (Reward compatibility). Rewards are action-local after any necessary state augmentation, and quotient rewards are trained or estimated as direct-image rewards on action cells, as in Proposition 4.3.2.

Assumption 4.4.3 (Liftability). Every coarse decision sequence used by the tower controller has at least one executable lift at the next finer tier. Moreover, the controller has access to a refinement routine that searches only inside the corresponding lift fiber.

Assumption 4.4.4 (Balanced addressability). There are positive integers $b_N, b_{N-1}, \ldots, b_0$ and nonnegative residual terms ϵ_i such that I think having a reference for the justification for $U_i^{\mathcal{S}}$ and $U_i^{\mathcal{A}}$ would be helpful in convincing the logarithmic growth. I am not debating the existence of the numbers, though. The other three assumptions are reasonable, but I'm having a hard time wrapping my head around the logarithm. And understanding this is probably crucial to the next phase of the project. I agree. I need to go through and make appropriate changes. the tower controller can choose the coarsest address and each subsequent refinement with expected cost

$$O(\log b_i) + \epsilon_i,$$

and the induced address capacity satisfies

$$\prod_{i=0}^{N} b_i \ge |\Omega|.$$

When the tower is approximately balanced, the product is of the same order as $|\Omega|$.

The numbers b_i are not necessarily graph degrees. They are effective address branching factors for the path/refinement problem. In practice, they should be estimated from tower fibers, policy supports, reward variance, and lift success rates.

Theorem 4.4.5 (Path-volume address theorem for quotient-tower HRL). *Let H be a finite hidden state/action graph, let $K < \infty$, and let*

$$G_t^0 \twoheadrightarrow G_t^1 \twoheadrightarrow \cdots \twoheadrightarrow G_t^N$$

be the quotient tower represented by the nested state/action partition tables produced by Algorithm 1. Let $\Omega \subseteq \mathbf{Path}_{\le K}(G_t^0)$ be a finite reward-labelled path set. Suppose Assumptions 4.4.1, 4.4.2, 4.4.3, and 4.4.4 hold. Then flat search over Ω has enumeration cost

$$\mathrm{Cost}_{\mathrm{flat}}(\Omega) = \Theta(|\Omega|) = \Theta(\mathcal{P}\text{-Vol}_\Omega),$$

where $\mathcal{P}\text{-Vol}_\Omega := |\Omega|$ is the finite path-volume of the relevant path set. By contrast, the tower controller has search cost bounded by

$$\mathrm{Cost}_{\mathcal{T}}(\Omega) \le C \sum_{i=0}^{N} \log b_i + \sum_{i=0}^{N} \epsilon_i$$

for a model-dependent constant C. In the balanced case, where $\prod_i b_i \asymp |\Omega|$, this becomes

$$\mathrm{Cost}_{\mathcal{T}}(\Omega) = O\big(\log \mathcal{P}\text{-Vol}_\Omega\big) + \sum_{i=0}^{N} \epsilon_i.$$

Without balance, the tower cost is controlled by the multilevel address length $\sum_i \log b_i$ plus residual lift/refinement errors.

Proof. For each tier i, the maps $\pi_i^{\mathcal{S}}$ and $\pi_i^{\mathcal{A}}$ induce a path-address map $\pi_{i,*} : \Omega \to \Omega_i$. Thus the tower induces nested partitions of Ω: two fine histories are equivalent at tier i if they have the same tier-i address. Selecting a fine history can therefore be decomposed into selecting a coarser address, then selecting a refinement inside the lift fiber above it, and so on until tier 0 is reached.

By Assumption 4.4.3, every selected coarse address used by the controller has an executable refinement to the next lower tier. By Assumption 4.4.4, the expected cost of selecting the ith address/refinement digit is bounded by $O(\log b_i) + \epsilon_i$. Summing over tiers gives

$$\mathrm{Cost}_{\mathcal{T}}(\Omega) \le C \sum_{i=0}^{N} \log b_i + \sum_{i=0}^{N} \epsilon_i.$$

If the address capacity is balanced, then

$$\sum_{i=0}^{N} \log b_i = \log\left(\prod_{i=0}^{N} b_i\right) = \Theta(\log |\Omega|).$$

Since $|\Omega| = \mathcal{P}\text{-Vol}_\Omega$ by definition, the logarithmic bound follows. Reward compatibility is not needed for the combinatorial address bound itself; it ensures that the address/refinement decisions optimize the same expected cumulative reward as the fine-scale problem, up to the residual terms and stutter corrections already included. □

Corollary 4.4.6 (`state_collapser` target theorem). *If the runtime contractions produced by `state_collapser` generate a quotient tower satisfying coverage, action-local reward descent, liftability, and balanced addressability for the path set actually explored by training, then the package replaces the flat path-space search term $\Theta(\mathcal{P}\text{-Vol})$ by a quotient-tower search term of order*

$$O(\log \mathcal{P}\text{-Vol}) + \textit{lift/refinement/statistical residuals.}$$

Warning 4.4.7. What the theorem does not say. The theorem does not say that arbitrary contractions help. Bad contractions can increase reward variance inside fibers, destroy liftability, or create coarse decisions that are cheap but useless. The theorem says that a quotient tower is a valid route to logarithmic path-space reduction precisely when it behaves like a balanced executable address system for the reward-labelled histories that matter.

4.5 Training-time interpretation and diagnostics

Theorem 4.4.5 is a theorem about search structure. It becomes a training-time theorem only after additional statistical assumptions relating sample complexity, optimization dynamics, and approximation error to the tower search term.

Let $S_{\mathrm{flat}}(\delta)$ be the sample budget required for a flat learner to identify a policy whose value is within δ of a target value on Ω. Let $S_{\mathcal{T}}(\delta)$ be the corresponding sample budget for a tower learner. The theorem supports a training-speed hypothesis of the schematic form

$$S_{\mathcal{T}}(\delta) \lesssim A(\delta) \left(\sum_i \log b_i \right) + B(\delta) \sum_i \epsilon_i + \mathrm{Approx}_{\mathcal{T}},$$

where A and B depend on the learning algorithm, and $\mathrm{Approx}_{\mathcal{T}}$ measures approximation error introduced by quotient rewards, coarse policies, stutter conventions, and lift/refinement routines.

The package should therefore report not only return curves, but also tower diagnostics:

1. flat and policy-effective $\mathcal{P}$-Vol estimates;
2. quotient compression ratios $|\Omega_i|/|\Omega_{i-1}|$;
3. lift-fiber size and entropy;
4. reward variance inside quotient fibers;
5. lift success rate;
6. coarse-policy value error and fine-refinement residual;
7. wall-clock and sample-efficiency comparisons against a non-tower baseline.

These diagnostics are the measurable content of the theorem's hypotheses. If they fail, the theorem explains why the tower should not be expected to improve training.

4.6 Compatibility with the explicit algorithms

The transfer from the earlier mathematical draft to the present paper requires one adjustment: the theorem should not be read as relying on an abstract "equivalence relation hull" construction. The explicit algorithm in this paper builds the tower by opening tierwise state and action partition tables, coalescing cells according to the ordered base-edge contraction schema, and maintaining outgoing-action pointers from state-cells to action-cells. This is enough.

The induced maps $\pi_i^{\mathcal{S}}$ and $\pi_i^{\mathcal{A}}$ provide the quotient addresses used in the proof. The full-build algorithm gives the static tower over G_t^0, and Algorithm 2 explains why the runtime update is lighter online: when exploration adds only a small new vista, only the newly discovered base-tier edges need to be propagated through the existing partition layers.

Thus the algorithms do not invalidate the path-volume theorem. They sharpen it. They show that the quotient tower used in the theorem is concretely represented by coupled nested partitions over the base graph, with one explicit implementation convention for internal loops/stutters.

References

[AZHE26] Shadab Ahamed, Niloufar Zakariaei, Eldad Haber, and Moshe Eliasof. Multiscale training of convolutional neural networks. *Transactions on Machine Learning Research*, 2026.

[CDGP91] Philip Candelas, Xenia C. De La Ossa, Paul S. Green, and Linda Parkes. A pair of calabi-yau manifolds as an exactly soluble superconformal theory. *Nuclear Physics B*, 359(1):21–74, 1991.

[CLRS09] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3 edition, 2009.

[Die99] Thomas G. Dietterich. State abstraction in MAXQ hierarchical reinforcement learning. In S. A. Solla, T. K. Leen, and K.-R. Müller, editors, *Advances in Neural Information Processing Systems 12*. MIT Press, 1999.

[FGK+09] Pierre Fraigniaud, Cyril Gavoille, Adrian Kosowski, Emmanuelle Lebhar, and Zvi Lotker. Universal augmentation schemes for network navigability. *Theoretical Computer Science*, 410(21):1970–1981, 2009.

[Ful97] William Fulton. *Young Tableaux: With Applications to Representation Theory and Geometry*, volume 35 of *London Mathematical Society Student Texts*. Cambridge University Press, Cambridge, 1997.

[HBML22] Matthias Hutsebaut-Buysse, Kevin Mets, and Steven Latré. Hierarchical reinforcement learning: A survey and open research challenges. *Machine Learning and Knowledge Extraction*, 4(1):172–221, 2022.

[HPV99] Michel Habib, Christophe Paul, and Laurent Viennot. Partition refinement techniques: An interesting algorithmic tool kit. *International Journal of Foundations of Computer Science*, 10(2):147–170, 1999.

[HX19] Juncai He and Jinchao Xu. Mgnet: A unified framework of multigrid and convolutional neural network. *CoRR*, abs/1901.10415, 2019.

[HZRS15] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. *CoRR*, abs/1512.03385, 2015.

[Kat92] Sheldon Katz. Rational curves on calabi-yau threefolds, 1992.

[Kle00] Jon M. Kleinberg. Navigation in a small world. *Nature*, 406(6798):845—845, 2000.

[Kon95] M. Kontsevich. Enumeration of rational curves via torus actions, 1995.

[Mal24] Abdullah Naeem Malik. *Simplicial Methods in Graph Machine Learning*. Ph.d. dissertation, Florida State University, 2024.

[MY16] Yury A. Malkov and Dmitry A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *CoRR*, abs/1603.09320, 2016.

[PSTQ22] Shubham Pateria, Budhitama Subagdja, Ah-Hwee Tan, and Chai Quek. Hierarchical reinforcement learning: A comprehensive survey. *ACM Computing Surveys*, 54(5):1–35, 2022.

[PT87] Robert Paige and Robert E. Tarjan. Three partition refinement algorithms. *SIAM Journal on Computing*, 16(6):973–989, 1987.

[RFB15] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. *CoRR*, abs/1505.04597, 2015.

[SB18] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018.

[SPS99] Richard S. Sutton, Doina Precup, and Satinder Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112(1–2):181–211, 1999.

Algorithm 2 Incremental Tower Update Along Exploration

3.2.7. Require: A current decorated tower $G_t^0 \twoheadrightarrow G_t^1 \twoheadrightarrow \cdots \twoheadrightarrow G_t^N$ in memory, a realized move from s_t to a genuinely new state s_{t+1}, with updated discovered base graph $G_{t+1}^0 = (\mathcal{S}_{t+1}^0,\ \mathcal{A}_{t+1}^0)$.

Ensure: An updated tower $G_{t+1}^0 \twoheadrightarrow G_{t+1}^1 \twoheadrightarrow \cdots \twoheadrightarrow G_{t+1}^{N'}$ in the form of nested state/action coset partitions decorating G_{t+1}^0, together with tower morphisms $G_t^\bullet \longrightarrow G_{t+1}^\bullet$.

function UpdateTower($G_t^\bullet, s_t, s_{t+1}$)
    $\Delta\mathcal{A}_{t+1}^0 \leftarrow \mathcal{A}_{t+1}^0 \setminus \mathcal{A}_t^0$
    $\Delta\mathcal{S}_{t+1}^0 \leftarrow \mathcal{S}_{t+1}^0 \setminus \mathcal{S}_t^0$
    Regard the updated discovered graph in memory as: 1. the node table on $\mathcal{S}_{t+1}^0$, together with 2. the outgoing-edge table $\mathcal{A}_{t+1}^0(s)$ indexed by source node s.

Initialization:

    for $s \in \mathcal{S}_t^0$ **do**
        Keep the existing tier-0 state pointer from s to $[s]_0$.
        Replace the tier-0 action pointer at $[s]_0$ with new edge bucket $\mathrm{Out}_0([s]_0) \leftarrow \mathcal{A}_{t+1}^0(s)$.
    end for
    Initialize a new singleton tier-0 state-cell for the new state: $[s_{t+1}]_0 \leftarrow \{s_{t+1}\}$.
    Initialize its tier-0 outgoing-action cell: $\mathrm{Out}_0([s_{t+1}]_0) \leftarrow \mathcal{A}_{t+1}^0(s_{t+1})$.
    Initiate a tier-0 pointer from s_{t+1} to $[s_{t+1}]_0$.
    Initiate a tier-0 pointer from $[s_{t+1}]_0$ to $\mathrm{Out}_0([s_{t+1}]_0)$.
    G_{t+1}^0 in memory $\leftarrow$ node table on $\mathcal{S}_{t+1}^0$ wiht tier-0 state and action partition pointers.
    Determine partition $\Delta\Sigma_0^1, \Delta\Sigma_0^2, \ldots, \Delta\Sigma_0^d$ of the newly discovered edge set $\Delta\mathcal{A}_{t+1}^0$, where $\Delta\Sigma_0^i$ is the tier-i contraction list contributed by the new vista.

Inductive step:

    $i \leftarrow 1$
    while $i \leq d$ **do**
        Initialize the tier-i partition tables by carrying forward the tier-$(i-1)$ structure on all previously existing nodes and cells.
        for $a \xrightarrow{e} b \in \Delta\Sigma_0^i$ **do**
            Let $C_a \in \Pi_{i-1}^{\mathcal{S}}$ be the tier-$(i-1)$ state-cell containing a.
            Let $C_b \in \Pi_{i-1}^{\mathcal{S}}$ be the tier-$(i-1)$ state-cell containing b.
            if $C_a \neq C_b$ **then**
                Coalesce the state data into a new tier-i state-cell: $C_{ab} \leftarrow \mathrm{MergeStateCells}(C_a, C_b)$.
                Define associated tier-$(i-1)$ action cells: $D_a := \mathrm{Out}_{i-1}(C_a)$ and $D_b := \mathrm{Out}_{i-1}(C_b)$
                Coalesce into a new tier-i action cell: $D_{ab} \leftarrow \mathrm{MergeActionCells}(D_a, D_b)$.
                Remove from D_{ab} all actions whose source and target both lie in C_{ab}, recording their contribution according to the chosen stutter convention.
                for node in $C_a \cup C_b$ **do**
                    Initiate a tier-i pointer from that node to the new tier-i state-cell C_{ab}.
                end for
                Initiate a tier-i pointer from C_{ab} to the new tier-i outgoing action cell D_{ab}.
            end if
        end for
        for $s \in \mathcal{S}_{t+1}^0$ **do**
            if s received no new tier-i state pointer during the preceding loop **then**
                Keep its inherited tier-i state pointer from the carried-forward tier-$(i-1)$ structure.
            end if
            if the tier-i state-cell containing s received no new outgoing pointer during the preceding loop **then**
                Keep its inherited tier-i outgoing pointer from the carried-forward tier-$(i-1)$ structure.
            end if